Robotics Laboratory 2025–2026: Final Report

Name: Ali
Surname: Shavakhi
Group Number: 03

Setup Description and Experiments Preparation

The final activity was carried out using a DEI-made TurtleBot mobile robot. The platform is a differential-drive robot with two independently actuated fixed wheels and one passive caster wheel used for mechanical support. The robot includes a Raspberry Pi 3B and an STM32 Nucleo board as computational units. The available sensing system consists of left and right wheel encoders, an inertial measurement unit (IMU), a LiDAR sensor, and an external Vicon motion-capture system. Encoder and IMU measurements were used for odometry and localization, while Vicon measurements were used either as feedback when available or as ground truth for post-processing and performance evaluation.

The robot was controlled through a ROS-based architecture. Dedicated ROS nodes were used to exchange data between the TurtleBot, the laboratory PC, and MATLAB/Simulink. The high-level controller was implemented in Simulink, where the desired linear and angular velocity commands were computed. These commands were then converted into wheel-speed commands and sent to the physical robot through ROS topics, while sensor data were logged for offline analysis.

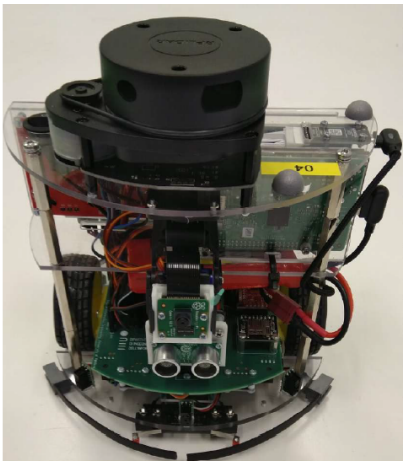


Figure 1: TurtleBot mobile robot.

Figure 1 shows the physical TurtleBot platform used in the experiments, including the mobile base, on-board electronics, and sensing devices.

The TurtleBot was modeled as a differential-drive robot (DDR). At the kinematic level, the DDR can be represented by the unicycle model. The robot pose in the planar Cartesian frame is described by

$$\mathbf{q} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T. \tag{1}$$

Here, x and y are the Cartesian coordinates of the robot position, and θ is the robot orientation measured counterclockwise with respect to the world-frame x-axis.

The control inputs of the unicycle model are the linear velocity v and the angular velocity ω.

Assuming pure rolling without slipping, the continuous-time kinematic model is

$$\dot{\mathbf{q}} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix}. \tag{2}$$

The relation between the wheel angular velocities and the unicycle inputs is

$$\begin{aligned} v &= \frac{r(\omega_R + \omega_L)}{2}, \\ \omega &= \frac{r(\omega_R - \omega_L)}{d}, \end{aligned} \tag{3}$$

where ω_L and ω_R are the left and right wheel angular velocities, r is the wheel radius, and d is the distance between the wheel centers. Equivalently, the commanded wheel velocities are obtained from v and ω as

$$\begin{aligned} \omega_L &= \frac{v}{r} - \frac{d\omega}{2r}, \\ \omega_R &= \frac{v}{r} + \frac{d\omega}{2r}. \end{aligned} \tag{4}$$

In this final activity, the wheel parameters were not re-identified. The nominal values

$$r = 0.03 \text{ m}, \qquad d = 0.165 \text{ m} \tag{5}$$

were used in the controller and simulation model. Therefore, these values must be interpreted as nominal geometric parameters, not as newly calibrated parameters for the final experiment. Possible calibration and parameter errors mainly appear as odometry drift, especially when localization relies only on encoder/IMU integration.

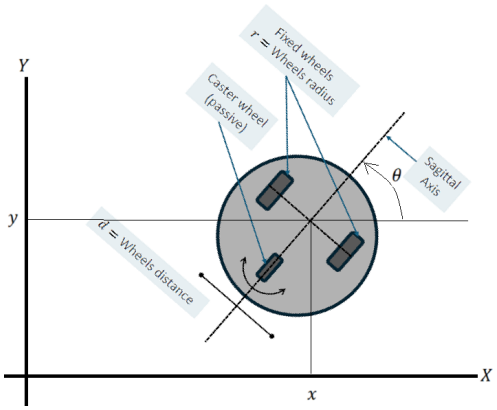


Figure 2: Kinematic representation of the differential-drive TurtleBot model.

Figure 2 defines the geometric quantities used in the DDR/unicycle equations, in particular the wheel radius r, the wheel distance d, the robot heading θ, and the world-frame coordinates (x, y).

Identification and calibration procedures had been considered in the previous experimental activities to analyze the sim-to-real gap and the effect of the physical parameters. However, no additional identification/calibration run was performed during the final activity. For this reason, the final report does not claim a new calibrated value of r or d. Instead, the EKF and Vicon-based feedback were used to reduce the influence of odometry drift and calibration uncertainty during closed-loop control and performance evaluation.

In the simulation environment, the robot was represented by the DDR/unicycle model implemented in Simulink. Therefore, when referring to the simulated robot, the pose is obtained by applying the

controller outputs v and ω to the simulated kinematic model. In the real test, the same type of velocity commands were converted into wheel-speed commands and sent to the physical TurtleBot through ROS.

For Tasks 1 and 2, the isolated simulations were initialized using the initial pose measured in the corresponding real run after the initial localization transient. This choice makes the comparison more meaningful, since both the simulated and real robot start from the same configuration. For Task 3, the final simulation was instead initialized from the final Task 2 configuration in order to evaluate the improved dedicated Task 3 controller. This difference is discussed explicitly in the Task 3 section.

The controller uses an estimated pose $\mathbf{q}_{\text{est}}$ as feedback. During the tests, this estimated pose was monitored together with the Vicon measurements and the odometry/IMU-based estimate. This diagnostic comparison is important because the odometry/IMU estimate accumulates drift over time, while Vicon provides an external pose reference when available. Therefore, the reliability of the feedback signal was evaluated by comparing $\mathbf{q}_{\text{est}}$, Vicon measurements, and the odometry/IMU estimate.

The performance of each task was evaluated using both trajectory-level and signal-level comparisons. The main regulation quantities were the position error and the orientation error with respect to the desired target configuration

$$\mathbf{q}_g = \begin{bmatrix} x_g & y_g & \theta_g \end{bmatrix}^T. \tag{6}$$

The planar position error was computed as

$$e_p(t) = \sqrt{(x_g - x(t))^2 + (y_g - y(t))^2}, \tag{7}$$

while the orientation error was computed as

$$e_\theta(t) = \left|\text{wrapToPi}\left(\theta_g - \theta(t)\right)\right|. \tag{8}$$

Final Activity Overview

The final activity was carried out on a common experimental map using the TurtleBot differential-drive platform. The tasks were first tested in simulation and then compared with the available real-robot data when applicable. The map includes the initial-position area, fixed obstacles, a cube, task reference points, and two regions with different Vicon measurement conditions.

Task 1 is a posture-regulation task toward the first target pose. Task 2 requires navigation to a new target while avoiding obstacles and crossing the red region, where the Vicon measurement is noisy. Task 3 consists of tracking a circular trajectory around the cube inside the orange region, where Vicon measurements can be frequently unavailable. Task 4 extends the same scenario by considering localization without direct motion-capture feedback and was evaluated in simulation.

In the red region, the Vicon signal is corrupted by Gaussian noise with standard deviation $\sigma_e = 0.02$. In the orange region, Vicon measurements are frequently unavailable, with loss probability $p_{\text{loss}} = 0.9$. The wheel-speed limit was $|\omega_L|, |\omega_R| \leq 10$ rad/s, and the sampling time was $T_s = 0.04$ s.

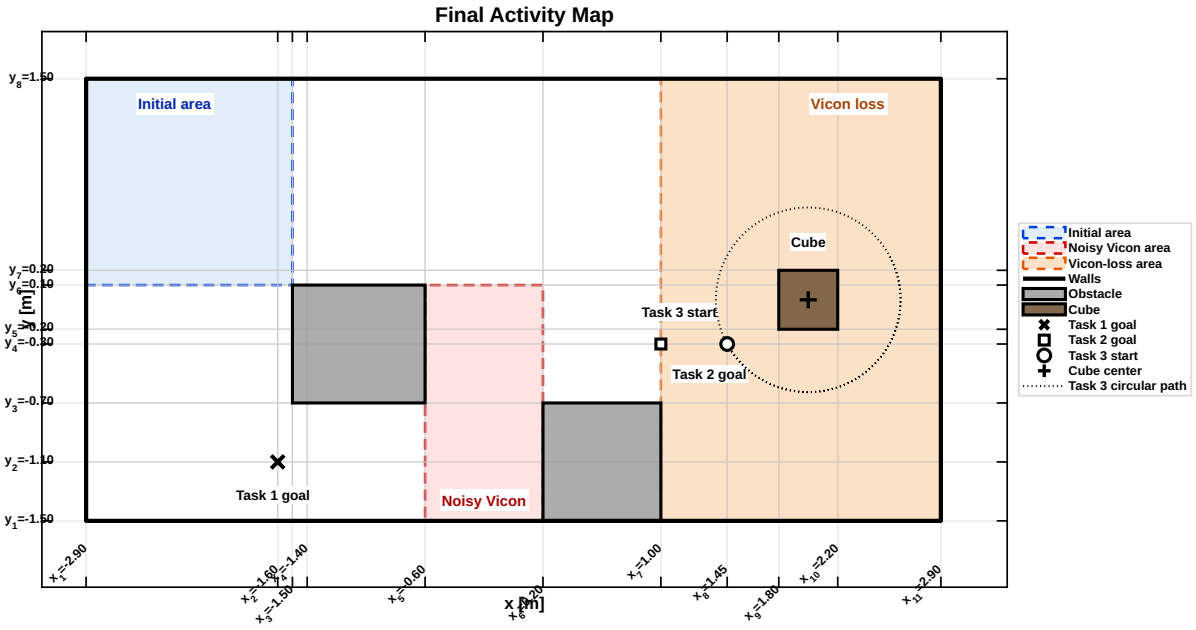


Figure 3: Experimental map and task reference points.

Figure 3 shows the common map used in the final activity. The figure indicates the initial area, task reference points, fixed obstacles, cube, and sensing-disturbance regions used in the following tasks.

Task 1

The objective of Task 1 was to move the TurtleBot from the initial-position area to the first target configuration

$$q_{\text{goal},1} = \begin{bmatrix} -1.60 & -1.10 & 0 \end{bmatrix}^T. \tag{9}$$

According to the official final-activity specification, Task 1 had to be tested from four different initial configurations inside the initial-position area. In the submitted real logs, three Task 1 real tests were available and were used to evaluate the robustness of the posture-regulation strategy with respect to different initial conditions. The detailed real/simulation comparison shown in Figures 5–9 corresponds to Run 3, which was used as the representative run for the time-domain analysis.

For this representative comparison, the initial pose was extracted from the first valid Vicon ground-truth sample inside the official initial-position area:

$$q_0 = \begin{bmatrix} -2.6852 & 0.4125 & -1.4487 \end{bmatrix}^T. \tag{10}$$

The isolated simulation was initialized with the same pose, so the real robot and the simulated model start from the same measured configuration. The Vicon signal was used as ground truth only for evaluation, while the controller used the online localizer estimate as feedback.

The available Task 1 initial-condition tests are summarized in Table 1. Run 3 is used as the representative real/simulation comparison for the detailed plots, while Runs 1 and 2 are reported numerically to keep the section compact without repeating the full set of figures for each run.

Table 1: Task 1 initial-condition tests and final real-run errors.

Run	x_0 [m]	y_0 [m]	θ_0 [rad]	Final e_p [m]	Final e_θ [rad]	Success/comment
1	-2.5548	0.5352	-1.5599	0.0478	0.5686	Position threshold reached; orientation threshold not reached.
2	-1.7048	1.0135	-1.5607	0.0491	0.9931	Position threshold reached; orientation threshold not reached.
3	-2.6852	0.4125	-1.4487	0.0473	0.4670	Position threshold reached; orientation threshold not reached.

The three available real tests show consistent position convergence to the Task 1 target region, since all final position errors are below the 0.05 m threshold. However, none of the real tests reached the orientation threshold within the available real-test duration. Therefore, the real data support successful position regulation from the available initial conditions, while full posture convergence is demonstrated only by the extended simulation of the representative run.

The task was implemented as a posture-regulation problem. Let

$$\Delta x = x_g - \hat{x}, \qquad \Delta y = y_g - \hat{y}, \qquad \rho = \sqrt{\Delta x^2 + \Delta y^2},$$

where $\hat{q} = [\hat{x}, \hat{y}, \hat{\theta}]^T$ is the pose estimate used by the controller. The implemented polar variables were

$$\theta_t = \operatorname{atan2}(\Delta y, \Delta x), \qquad \gamma = \operatorname{wrapToPi}(\theta_t - \hat{\theta}),$$
$$\delta = \operatorname{wrapToPi}(\gamma + \hat{\theta} - \theta_g).$$

The unicycle commands were computed as

$$v = k_1 \rho \cos\gamma, \tag{11}$$

$$\omega = k_2 \gamma + k_1 \frac{\sin\gamma}{\gamma} \cos\gamma(\gamma + k_\theta \delta), \tag{12}$$

with $\sin\gamma/\gamma = 1$ for very small $|\gamma|$. The numerical gains used in the experiment were

$$k_1 = 0.5, \qquad k_2 = 1.2, \qquad k_\theta = 0.3.$$

Close to the target position, when $\rho < 0.001$ m, the controller sets $v = 0$ and uses a pure rotation command

$$\omega = k_\theta e_\theta, \qquad e_\theta = \operatorname{wrapToPi}(\theta_g - \hat{\theta}).$$

The final rotation is stopped when $|e_\theta| < 0.05$ rad.

The unicycle commands were converted into differential-drive wheel commands by

$$\omega_L = \frac{v}{r} - \frac{d\omega}{2r}, \qquad \omega_R = \frac{v}{r} + \frac{d\omega}{2r}, \tag{13}$$

with $r = 0.03$ m, $d = 0.165$ m, and $T_s = 0.04$ s. The wheel-speed saturation was

$$|\omega_L|, |\omega_R| \leq 10 \text{ rad/s}. \tag{14}$$

This was the main actuation constraint checked in the real experiment.

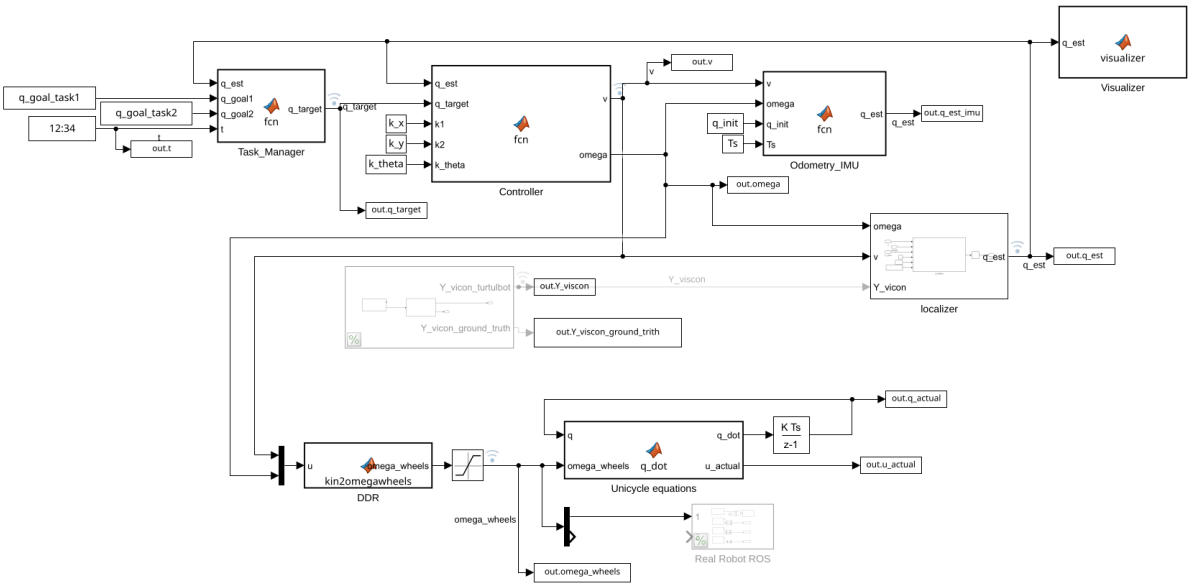


Figure 4: Task 1 Simulink implementation.

Figure 4 shows the implementation used for both the actual test and the isolated simulation. The model includes the localizer, posture-regulation controller, differential-drive conversion, real robot/ROS interface, simulated unicycle model, and logging blocks. The same controller computes v and ω from the estimated pose and the active target. The DDR block then converts these commands into wheel angular velocities. This figure documents the complete signal path used to generate the commands and to record the data analyzed in the following plots.

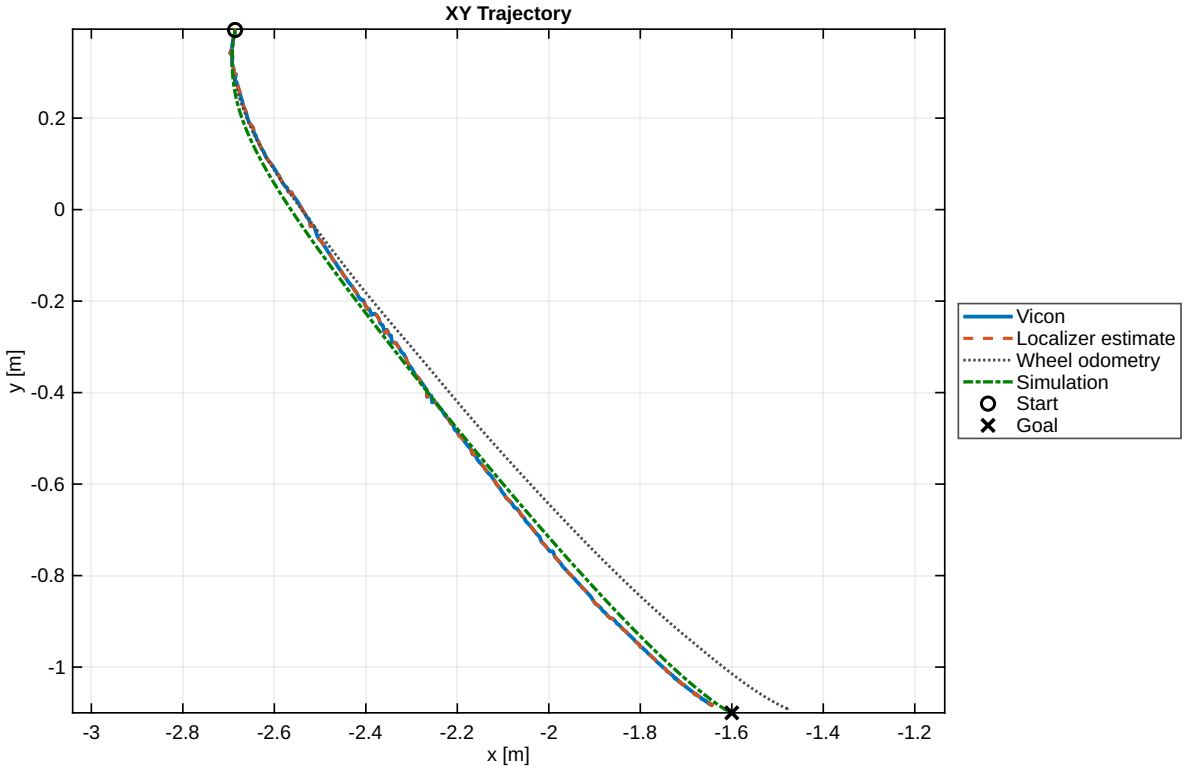


Figure 5: Task 1 x-y trajectory.

Figure 5 verifies the planar convergence of the representative run. It compares the Vicon ground truth, localizer estimate, wheel odometry, and simulation, together with the start and goal points.

The real robot moved from the initial pose toward $q_{\text{goal},1}$ and reached the target region. The Vicon trajectory and the localizer estimate remain close, showing that the feedback pose was consistent with the ground truth. Wheel odometry follows the same qualitative path but drifts because it is obtained by integrating wheel-speed measurements without absolute correction. The simulation reproduces the same qualitative motion toward the goal, with remaining differences mainly due to model assumptions, sampling, and low-level actuation dynamics.

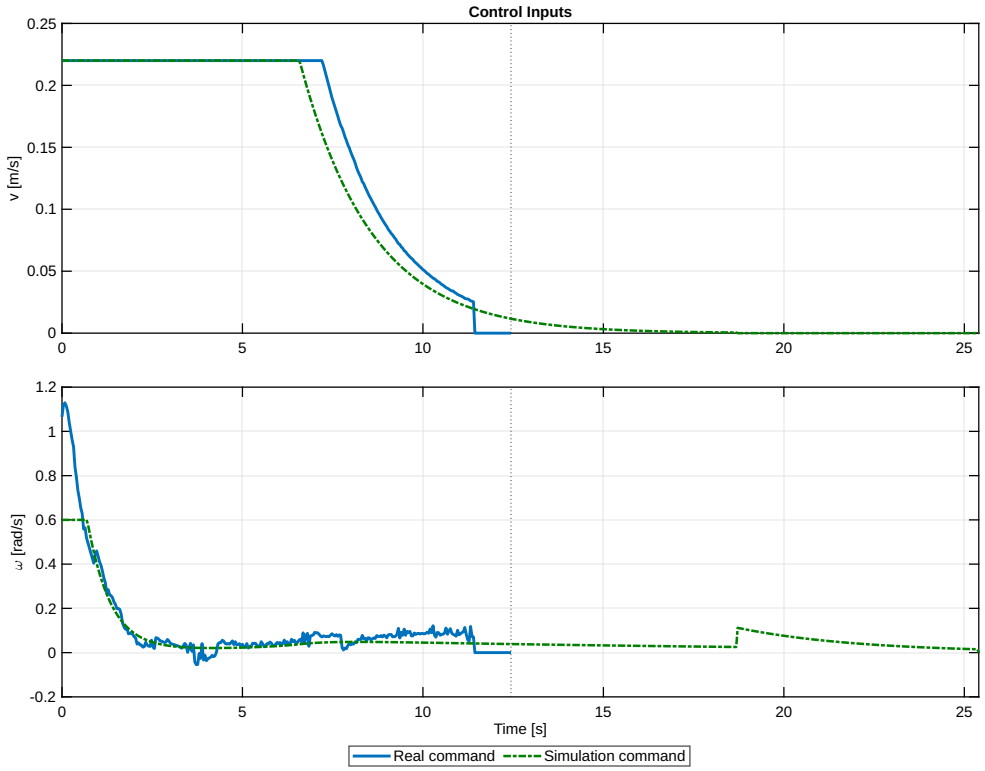


Figure 6: Task 1 control inputs.

Figure 6 explains the behavior of the commanded linear and angular velocities. The linear velocity initially reaches the imposed maximum value $v_{\max} = 0.22$ m/s, then decreases as the robot approaches the goal. The angular velocity is larger at the beginning, where the robot must align toward the target direction, and becomes smaller during the final approach. The simulation continues after the real-test interval in order to evaluate the remaining orientation convergence.

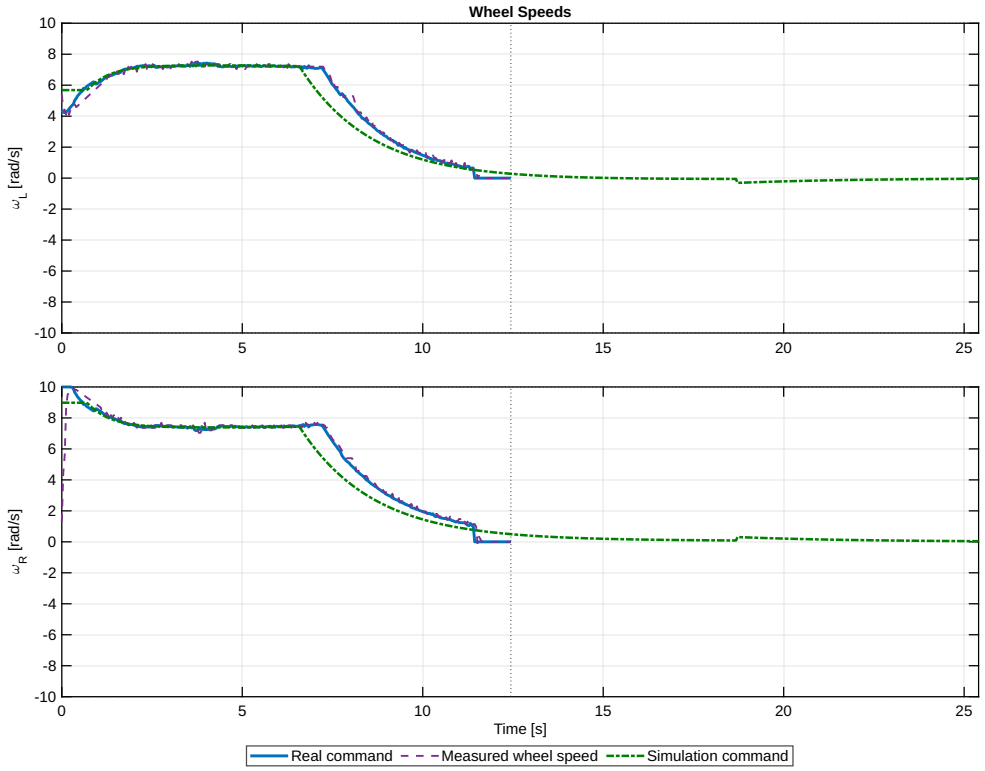


Figure 7: Task 1 wheel angular velocities.

Figure 7 verifies the wheel-speed saturation. The maximum commanded wheel speed was 10.00 rad/s, while the maximum measured wheel speed was 9.98 rad/s. Therefore, the wheel-speed limit was respected during the real test. Apart from the short initial transient, the measured wheel speeds follow the commanded values closely. The simulation wheel commands also remain within the same admissible range.

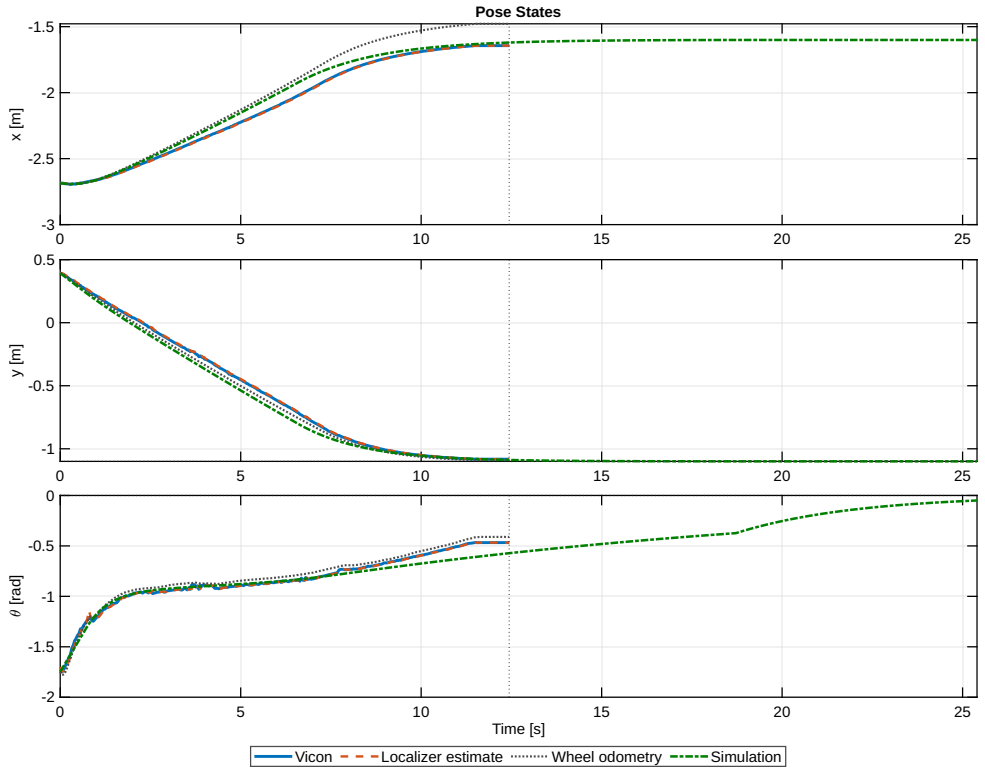


Figure 8: Task 1 pose states.

Figure 8 shows the time evolution of $x(t)$, $y(t)$, and $\theta(t)$ for the representative run and the corresponding simulation. The real x and y coordinates converge toward the desired target coordinates -1.60 m and -1.10 m. The localizer remains close to Vicon, while wheel odometry accumulates a small integration error. The simulation shows the same qualitative convergence trend. The orientation does not fully reach $\theta_g = 0$ before the real test ends; the extended simulation continues after the real-test interval and reduces the remaining heading error.

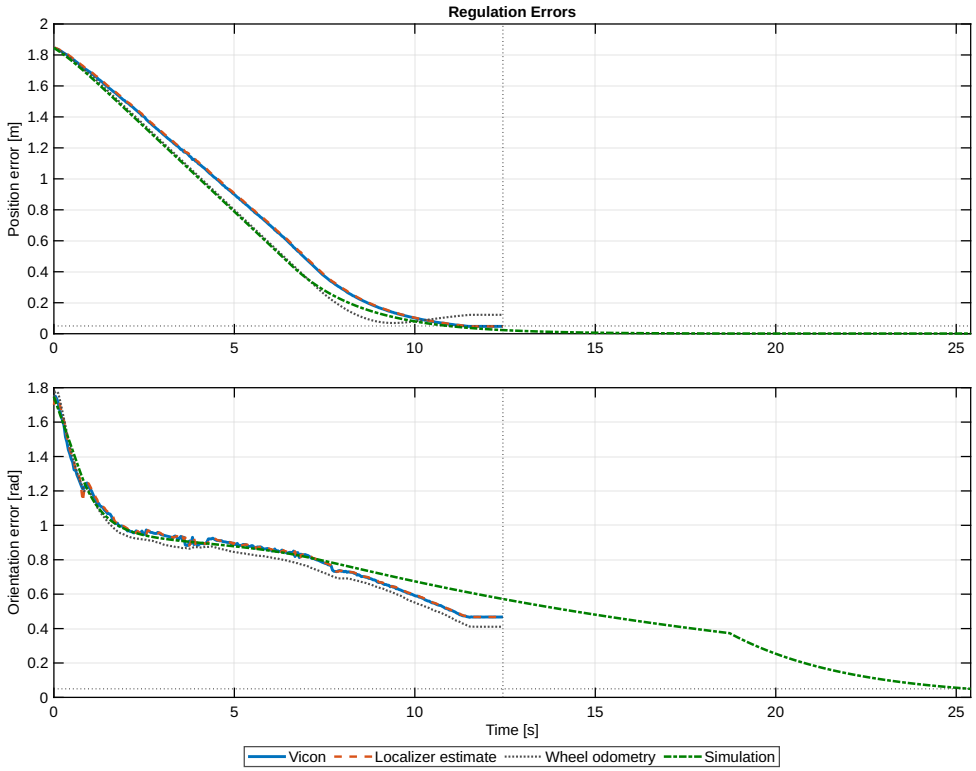


Figure 9: Task 1 regulation errors.

Figure 9 gives the quantitative regulation result for the representative run. At the end of the real experiment, $t = 12.44$ s, the Vicon position error was 0.0473 m, below the 0.05 m threshold. The remaining orientation error was 0.467 rad, equal to 26.76°, so the orientation threshold was not reached within the available real-test duration. At the same time instant, the simulation position error was 0.0234 m, while its orientation error was 0.572 rad. When the simulation was continued, both thresholds were reached at 25.40 s, with a position error of 9.90×10^{-4} m and an orientation error of 0.0497 rad.

Overall, Task 1 was successful with respect to the position objective: the robot reached the target region in the available real runs, the localizer estimate was consistent with the Vicon ground truth, and the wheel-speed limits were respected. Full posture convergence, including the final orientation threshold, was observed only in the extended simulation, not within the available real-test duration.

Task 2

The objective of Task 2 was to move the TurtleBot from the final configuration of Task 1 to

$$q_{\text{goal},2} = \begin{bmatrix} 1.00 & -0.30 & 0 \end{bmatrix}^T.$$

The reported Task 2 segment was initialized from the first valid Vicon ground-truth sample of the Task 2 log:

$$q_{0,2} = \begin{bmatrix} -1.6439 & -1.0826 & -0.4674 \end{bmatrix}^T.$$

Compared with Task 1, this task also required obstacle avoidance and crossing the red region of the map, where the Vicon measurements are noisier. Therefore, a waypoint-based posture-regulation strategy was used together with an EKF localizer.

The waypoint sequence was

$$q_{\mathrm{wp},1} = \begin{bmatrix} -0.20 & -1.10 & 0 \end{bmatrix}^T, \qquad q_{\mathrm{wp},2} = \begin{bmatrix} -0.20 & -0.30 & \pi/2 \end{bmatrix}^T, \qquad q_{\mathrm{goal},2}.$$

The first waypoint keeps the robot below the upper obstacle. The second waypoint guides the robot through the central corridor and controls the entry through the red noisy-Vicon region before the final motion toward the goal. The finite-state manager selected the active target using the current EKF pose estimate. Its switching logic is summarized below, where $p = [x\ y]^T$.

State 1	$q_{\mathrm{target}} = q_{\mathrm{wp},1}$	$\|p - p_{\mathrm{wp},1}\| < 0.10$ m $\Rightarrow$ **State 2**
State 2	$q_{\mathrm{target}} = q_{\mathrm{wp},2}$	$\|p - p_{\mathrm{wp},2}\| < 0.08$ m and $\|\tilde{\theta}_{\mathrm{wp},2}\| < 0.45$ rad $\Rightarrow$ **State 3**
State 3	$q_{\mathrm{target}} = q_{\mathrm{goal},2}$	$\|p - p_{\mathrm{goal},2}\| < 0.05$ m $\Rightarrow$ hold final target.

A safety timeout of 4.0 s was also included at the second waypoint to avoid remaining indefinitely in State 2 if the heading condition was not reached. The same posture-regulation controller used in Task 1 generated the unicycle inputs v and ω. These commands were converted into wheel angular velocities using the differential-drive relation introduced in the setup section, with command saturation

$$|\omega_L|,\ |\omega_R| \le 10 \text{ rad/s}.$$

For localization, the controller did not use the raw Vicon measurement directly as feedback. Instead, the feedback pose was provided by an Extended Kalman Filter with state and input

$$\hat{q}_k = \begin{bmatrix} \hat{x}_k & \hat{y}_k & \hat{\theta}_k \end{bmatrix}^T, \qquad u_k = \begin{bmatrix} v_k & \omega_k \end{bmatrix}^T.$$

The sampling time was $T_s = 0.04$ s. The EKF prediction step used the Euler-discretized unicycle model

$$\hat{q}_k^- = \hat{q}_{k-1}^+ + T_s \begin{bmatrix} v_k \cos\hat{\theta}_{k-1}^+ \\ v_k \sin\hat{\theta}_{k-1}^+ \\ \omega_k \end{bmatrix}.$$

The covariance prediction was

$$P_k^- = F_k P_{k-1}^+ F_k^T + Q,$$

where

$$F_k = \begin{bmatrix} 1 & 0 & -v_k T_s \sin\hat{\theta}_{k-1}^+ \\ 0 & 1 & v_k T_s \cos\hat{\theta}_{k-1}^+ \\ 0 & 0 & 1 \end{bmatrix}.$$

When a valid Vicon measurement was available, the correction step used

$$y_k = H q_k + r_k, \qquad H = I_3, \qquad r_k \sim \mathcal{N}(0, R),$$

with

$$y_k = \begin{bmatrix} x_{\mathrm{Vicon},k} & y_{\mathrm{Vicon},k} & \theta_{\mathrm{Vicon},k} \end{bmatrix}^T.$$

The complete EKF correction equations were

$$\tilde{y}_k = y_k - H\hat{q}_k^-, \qquad S_k = H P_k^- H^T + R,$$

$$K_k = P_k^- H^T S_k^{-1}, \qquad \hat{q}_k^+ = \hat{q}_k^- + K_k \tilde{y}_k,$$

$$P_k^+ = (I_3 - K_k H)P_k^-.$$

The orientation component of the innovation was wrapped to $(-\pi, \pi]$. If the Vicon output was $[0, 0, 0]^T$, the correction step was skipped and only the prediction was used. In the extracted Task 2 data, Vicon loss was $0/509 = 0.00\%$, so the main disturbance in this task was noisy Vicon in the red region rather than measurement dropout. The IMU was not used as an additional EKF correction measurement in this Task 2 localizer; it was used only for odometry/diagnostic comparison.

EKF parameter	**Value**
P_0	$0.1I_3$
Q	$\mathrm{diag}(10^{-4}, 10^{-4}, 5 \cdot 10^{-4})$
R_{normal}	$\mathrm{diag}(10^{-5}, 10^{-5}, 5 \cdot 10^{-5})$
R_{red}	$\mathrm{diag}(0.7, 0.7, 0.9)$
T_s	0.04 s

Inside the red region, R_{red} is much larger than R_{normal}. This reduces the weight of the Vicon correction when the robot is inside the noisy region, so the EKF relies more on the unicycle prediction there and returns to a stronger Vicon correction outside the red region.

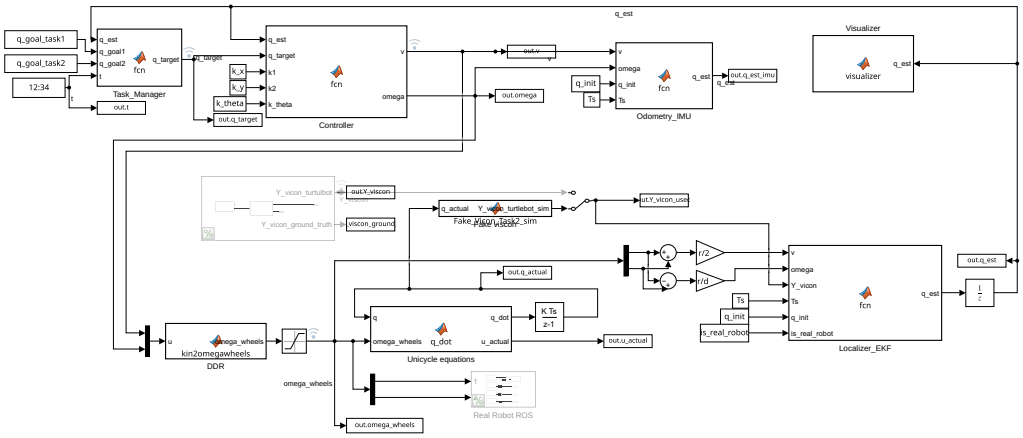


Figure 10: Task 2 Simulink model.

Figure 10 shows the implemented Simulink structure, including the waypoint manager, posture-regulation controller, differential-drive conversion, simulated robot model, Vicon block, and EKF localizer. The diagram confirms that the controller feedback is the estimated pose, not the raw Vicon signal.

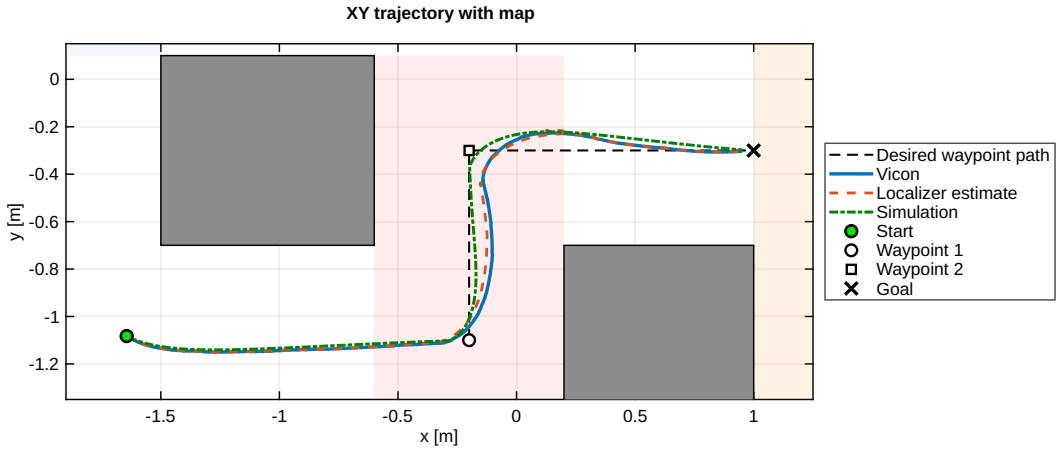


Figure 11: x-y trajectory with map.

Figure 11 compares the desired waypoint path, Vicon trajectory, EKF/localizer estimate, and simulation in the map. The desired waypoint path guides the robot through the safe corridor instead of moving directly to the goal. The real Vicon trajectory follows the planned corridor, avoids the obstacles, crosses the red noisy-Vicon region, and reaches the goal area. The EKF estimate remains close to Vicon, showing that the feedback pose is consistent with the measured real trajectory. The simulation follows the same qualitative path, with small differences near waypoint transitions, likely due to the nominal kinematic model, real actuator dynamics, measurement noise, and wheel-ground effects.

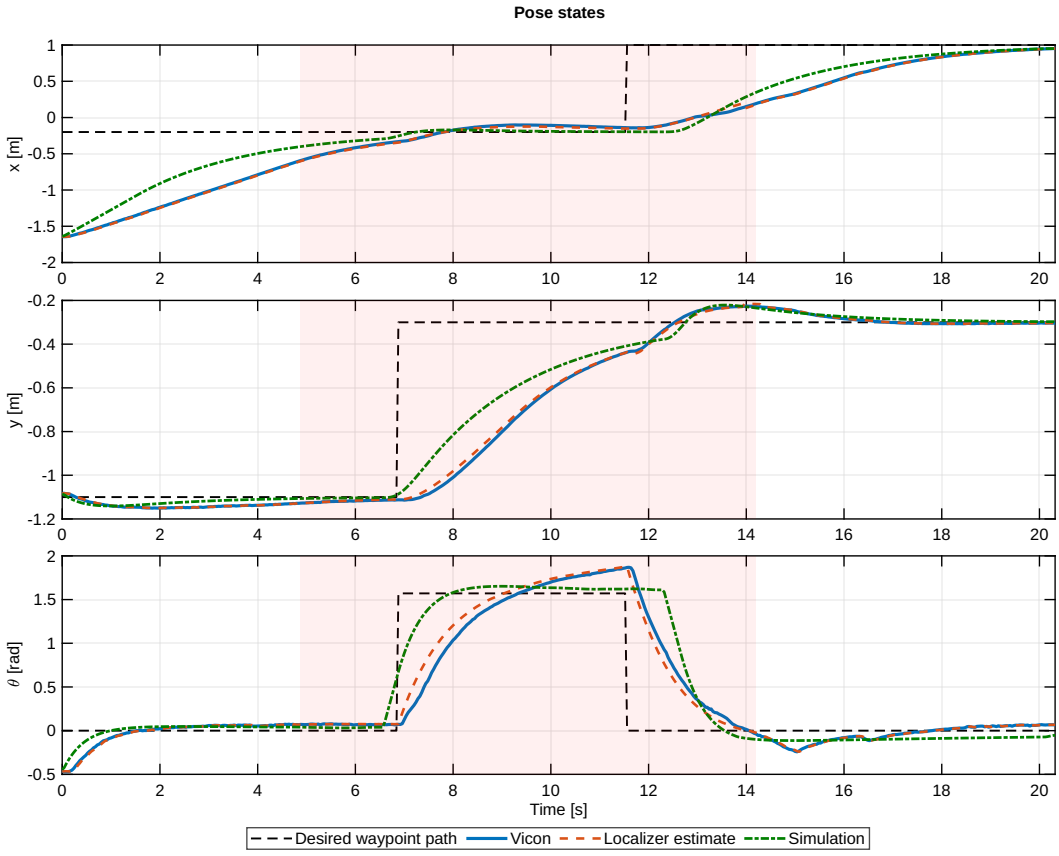


Figure 12: Pose states.

Figure 12 compares the desired waypoint signals, Vicon measurement, EKF/localizer estimate, and simulation with respect to time for $x(t)$, $y(t)$, and $\theta(t)$. The desired signal shows the waypoint

changes, while the Vicon and EKF curves remain close during most of the experiment. The largest differences appear while crossing the red interval, where the EKF intentionally gives less weight to the noisy Vicon measurement. The simulation is slightly shifted with respect to the real robot near waypoint transitions. This is expected because the isolated simulation was exported over a longer interval, while the simulated curves in the comparison plots were resampled on the experimental time vector.

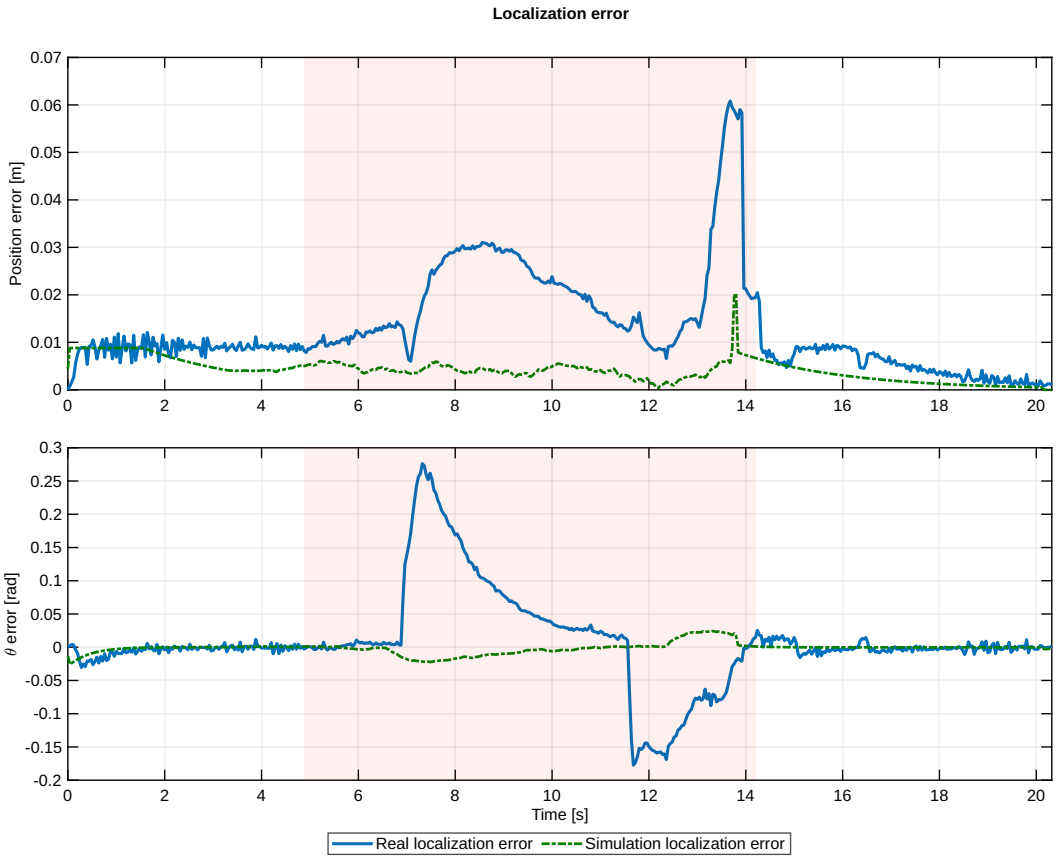


Figure 13: Localization error.

Figure 13 compares the EKF estimate versus Vicon in the real test and the simulated EKF estimate versus the simulated true pose in simulation. The real position RMSE was 0.0168 m, while the maximum real position error was 0.0608 m. Inside the red region, the real RMSE increased to 0.0235 m, and the maximum error still occurred in this interval. This confirms that the red region was the most critical part for localization. However, the error remained bounded and decreased after leaving the red region. In simulation, the localization RMSE was 0.0049 m, with a maximum error of 0.0244 m, because the simulated model and measurement process are more controlled than the real platform.

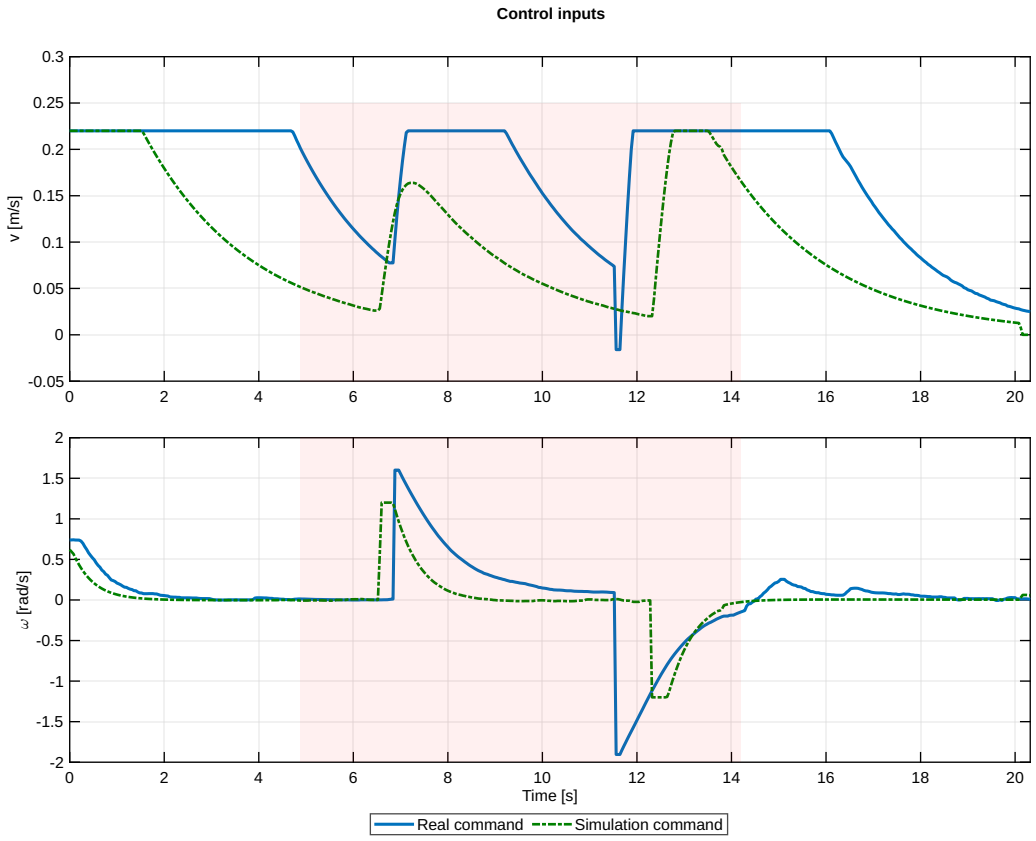


Figure 14: Control inputs.

Figure 14 shows the commanded linear and angular velocities. The real linear velocity reached 0.220 m/s, while the maximum real angular velocity was 2.021 rad/s. The linear velocity is high during the straight parts and decreases near waypoint transitions and close to the final target. The angular velocity is mainly activated during the turns required to align the robot with the next path segment. The simulation reaches the same linear-velocity limit, while its maximum angular velocity was 1.200 rad/s. The sharper real variations near the waypoint transitions are likely due to measurement noise, sampling effects, and low-level actuator response.

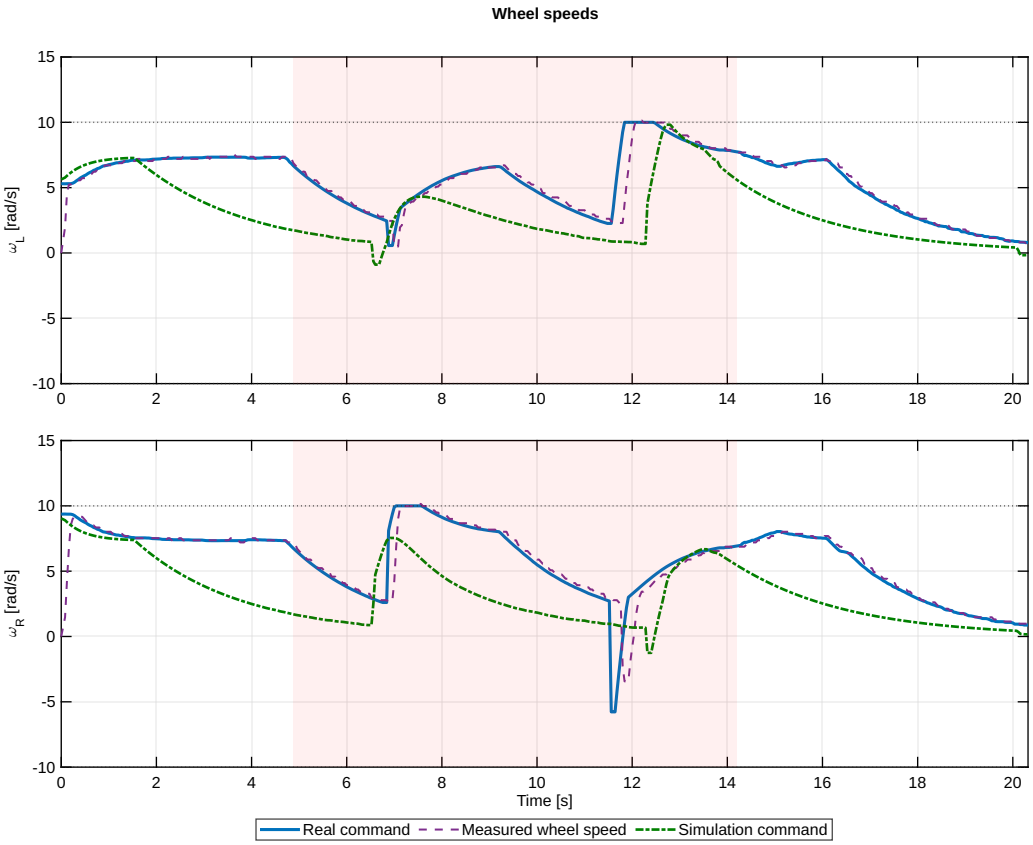


Figure 15: Wheel angular velocities.

Figure 15 compares the commanded wheel speeds, measured wheel speeds, and simulated wheel commands. The commanded wheel speeds reached the imposed command limit of 10.00 rad/s, so the command-level saturation was active but respected. The measured wheel speeds followed the commanded values with small deviations and a slight overshoot. The maximum measured wheel speed was 10.472 rad/s, which is slightly above the nominal command limit and is likely due to low-level motor dynamics and measurement noise. The maximum simulated wheel command was 9.968 rad/s, remaining within the same admissible range.

At the end of the real experiment, the final Vicon pose was

$$q_{\text{Vicon},f} = \begin{bmatrix} 0.9523 & -0.3038 & 0.0662 \end{bmatrix}^T.$$

The final Vicon position error with respect to the Task 2 goal was 0.0478 m, below the 0.05 m target threshold. The final orientation error was 0.0662 rad, corresponding to 3.79°. The simulation also reached the target region, with final position error 0.0490 m and final orientation error 0.0513 rad. Overall, Task 2 was successful: the robot avoided the obstacles, crossed the noisy Vicon region, kept a bounded EKF localization error, respected the command-level wheel-speed limit, and reached the final Cartesian target.

Task 3

The objective of Task 3 was to perform a circular motion around the cube inside the orange Vicon-loss region. Since the setup section already introduced the differential-drive model, the Vicon-loss rule, and the EKF structure, this section focuses on the Task 3 reference generation, controller choice, and real/simulation interpretation.

A key point is that the available real run and the final simulation were not generated with exactly

the same Task 3 algorithm. The real experiment was performed with the earlier full-activity model, where Task 1, Task 2, and Task 3 were executed in one state machine. In that real model, after Task 2 the robot first moved to

$$q_{\text{wp3}} = \begin{bmatrix} 1.45 & -0.30 & 0 \end{bmatrix}^T,$$

then to the old Task 3 starting target

$$q_{\text{start,old}} = \begin{bmatrix} 2.00 & -0.60 & 0 \end{bmatrix}^T.$$

After a short waiting phase, the older circular target was generated with

$$R_{\text{old}} = 0.60 \text{ m}, \qquad \omega_{\text{old}} = 0.35 \text{ rad/s}.$$

The old real target generator used

$$x_d = x_{\text{start,old}} + R_{\text{old}} \sin(\omega_{\text{old}} t_e),$$

$$y_d = y_{\text{start,old}} + R_{\text{old}} \left(1 - \cos(\omega_{\text{old}} t_e)\right),$$

$$\theta_d = \text{wrapToPi}(\omega_{\text{old}} t_e),$$

where the elapsed time included a look-ahead term and was capped in the implementation. The real run therefore documents the behavior of this earlier full-task implementation.

The final simulation was then improved and implemented as a dedicated Task 3 model. It starts from the final Task 2 configuration,

$$q_{\text{goal,2}} = \begin{bmatrix} 1.00 & -0.30 & 0 \end{bmatrix}^T,$$

moves to the circular starting pose

$$q_{\text{start,3}} = \begin{bmatrix} 1.45 & -0.30 & \theta_{\text{start}} \end{bmatrix}^T,$$

and then tracks a continuous circular reference around the cube. The circle center is

$$p_c = \begin{bmatrix} 2.00 & 0.00 \end{bmatrix}^T,$$

so the radius is

$$R = \|p_{\text{start,3}} - p_c\| = \sqrt{(1.45 - 2.00)^2 + (-0.30 - 0.00)^2} = 0.6265 \text{ m}.$$

The desired motion consists of

$$N_{\text{turns}} = 1.5$$

turns with

$$\omega_c = 0.30 \text{ rad/s}.$$

The final simulation reference is

$$\alpha(t) = \alpha_0 + \omega_c t,$$

$$x_d(t) = x_c + R\cos(\alpha(t)), \qquad y_d(t) = y_c + R\sin(\alpha(t)),$$

$$\theta_d(t) = \text{wrapToPi}\left(\alpha(t) + \frac{\pi}{2}\right).$$

The nominal circular duration is

$$T_c = \frac{2\pi N_{\text{turns}}}{\omega_c} = \frac{2\pi(1.5)}{0.30} = 31.42 \text{ s}.$$

Unlike Task 2, the final Task 3 simulation is not a multi-waypoint path. The only intermediate target is the circular starting pose. After reaching this pose, the controller tracks the continuous circular reference. The final Task 3 manager uses three states:

State 1	$q_{\text{target}} = q_{\text{start},3}$	transfer to the circular starting pose,
State 2	$q_{\text{target}} = q_d(t)$	track the circular reference,
State 3	$q_{\text{target}} = q_{\text{hold}}$	stop and hold the final pose.

The transition to the circular phase is enabled only when

$$\|p - p_{\text{start},3}\| < 0.050 \text{ m}, \qquad |\tilde{\theta}_{\text{start}}| < 0.05 \text{ rad}.$$

This heading condition was added because starting the circular trajectory with a wrong orientation produces an initial cut or offset in the circle.

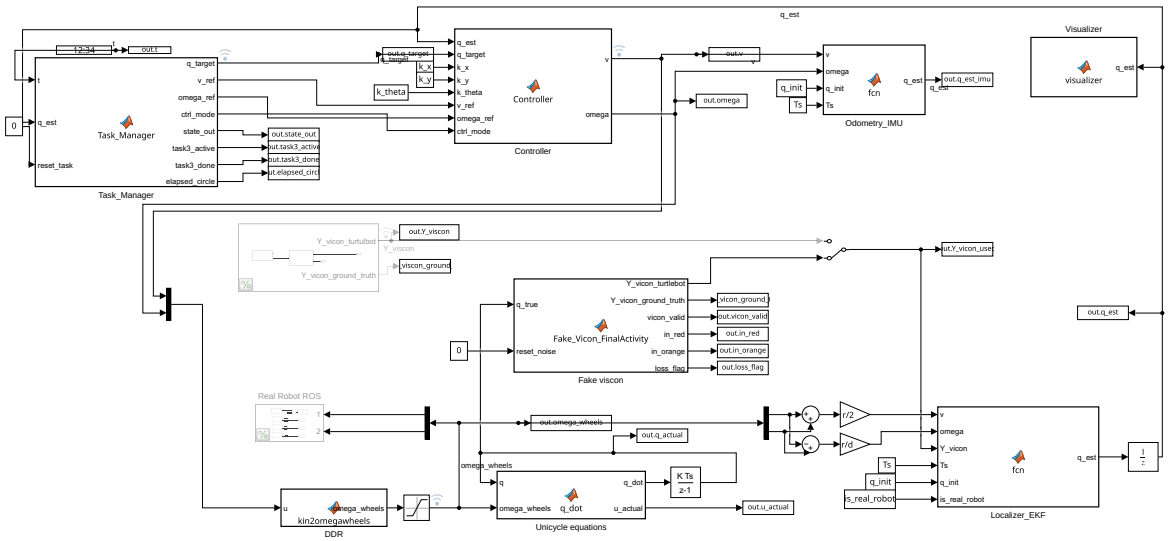


Figure 16: Task 3 Simulink implementation.

Figure 16 shows the final Task 3 simulation model. The task manager generates the active reference, reference velocities, and controller mode. The controller computes v and ω, the DDR block converts them into wheel-speed commands, the fake Vicon block applies the orange-region measurement-loss rule, and the EKF/localizer provides the feedback pose.

The controller theory is different in the two implementations. The real full-activity model used the same posture-regulation controller structure used in the previous tasks. This controller is suitable for reaching fixed target poses, but in the real run it was applied to a moving circular target. Therefore, the real target error should not be interpreted as the tracking error of the final tuned controller.

In the final simulation, the transfer phase still uses posture regulation to reach $q_{\text{start},3}$, but the circular phase uses nonlinear trajectory tracking. The tracking errors are expressed in the robot frame:

$$e_1 = \cos\theta(x_d - x) + \sin\theta(y_d - y),$$
$$e_2 = -\sin\theta(x_d - x) + \cos\theta(y_d - y),$$
$$e_3 = \text{wrapToPi}(\theta_d - \theta).$$

The final circular tracking commands are

$$v = v_r \cos(e_3) + k_1 e_1,$$
$$\omega = \omega_r + k_2 v_r \frac{\sin(e_3)}{e_3} e_2 + k_3 e_3,$$

with $\sin(e_3)/e_3 = 1$ for very small $|e_3|$. The tuned gains were

$$k_1 = 0.70, \qquad k_2 = 2.30, \qquad k_3 = 0.70.$$

The command saturations were

$$|v| \le 0.22 \text{ m/s}, \qquad |\omega| \le 0.60 \text{ rad/s}.$$

This nonlinear tracking law is more appropriate for the final Task 3 simulation because the desired pose changes continuously during the circular motion.

The controller uses the EKF estimate as feedback. Task 3 uses the same EKF covariance parameters reported for Task 2:

$$P_0 = 0.1 I_3,$$
$$Q = \text{diag}(10^{-4}, 10^{-4}, 5 \cdot 10^{-4}),$$
$$R_{\text{normal}} = \text{diag}(10^{-5}, 10^{-5}, 5 \cdot 10^{-5}),$$
$$R_{\text{red}} = \text{diag}(0.7, 0.7, 0.9).$$

In the orange region, Vicon can be lost with probability $p_{\text{loss}} = 0.90$. During Vicon loss, the EKF propagates the pose using the unicycle prediction. When a valid Vicon measurement becomes available, the EKF correction is applied. Therefore, Task 3 mainly tests robustness to measurement dropout rather than red-region measurement noise.

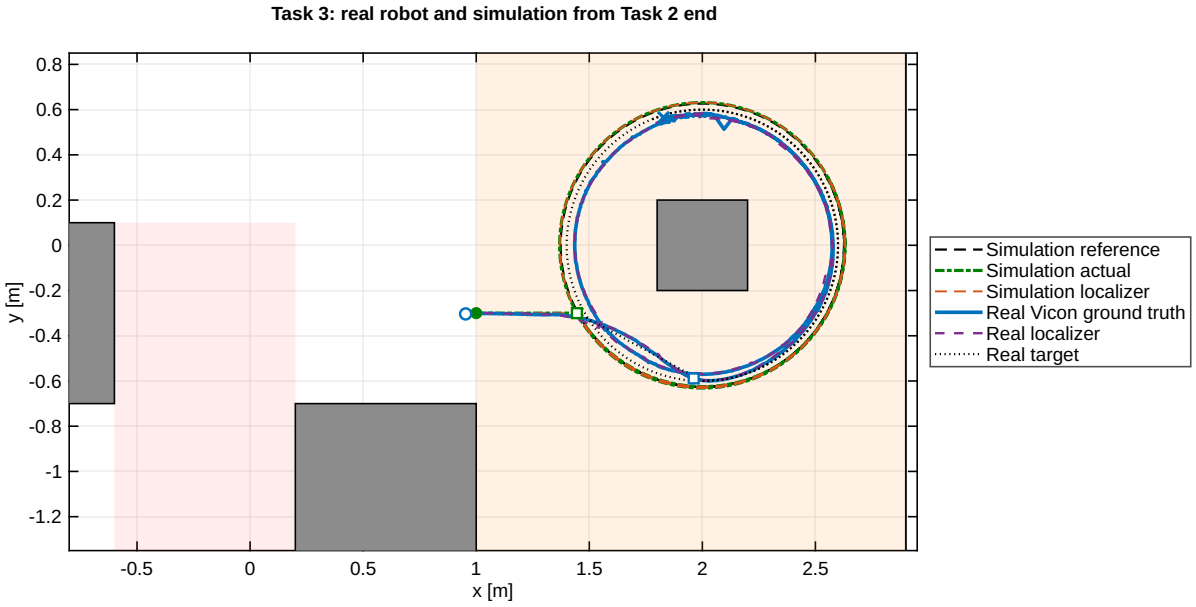


Figure 17: Task 3 $x - y$ trajectory.

Figure 17 gives a qualitative $x-y$ comparison between the available real run and the final simulation. In the final simulation, the reference, actual trajectory, and localizer estimate are almost overlapped, and the circular tracking RMSE is 0.0071 m. This validates the improved nonlinear circular tracking controller in simulation. The real Vicon trajectory also goes around the cube inside the orange region without collision, supporting the feasibility of the maneuver on the physical TurtleBot. The difference between the real target and the final simulated reference is expected because the real run used the earlier full-activity reference generator.

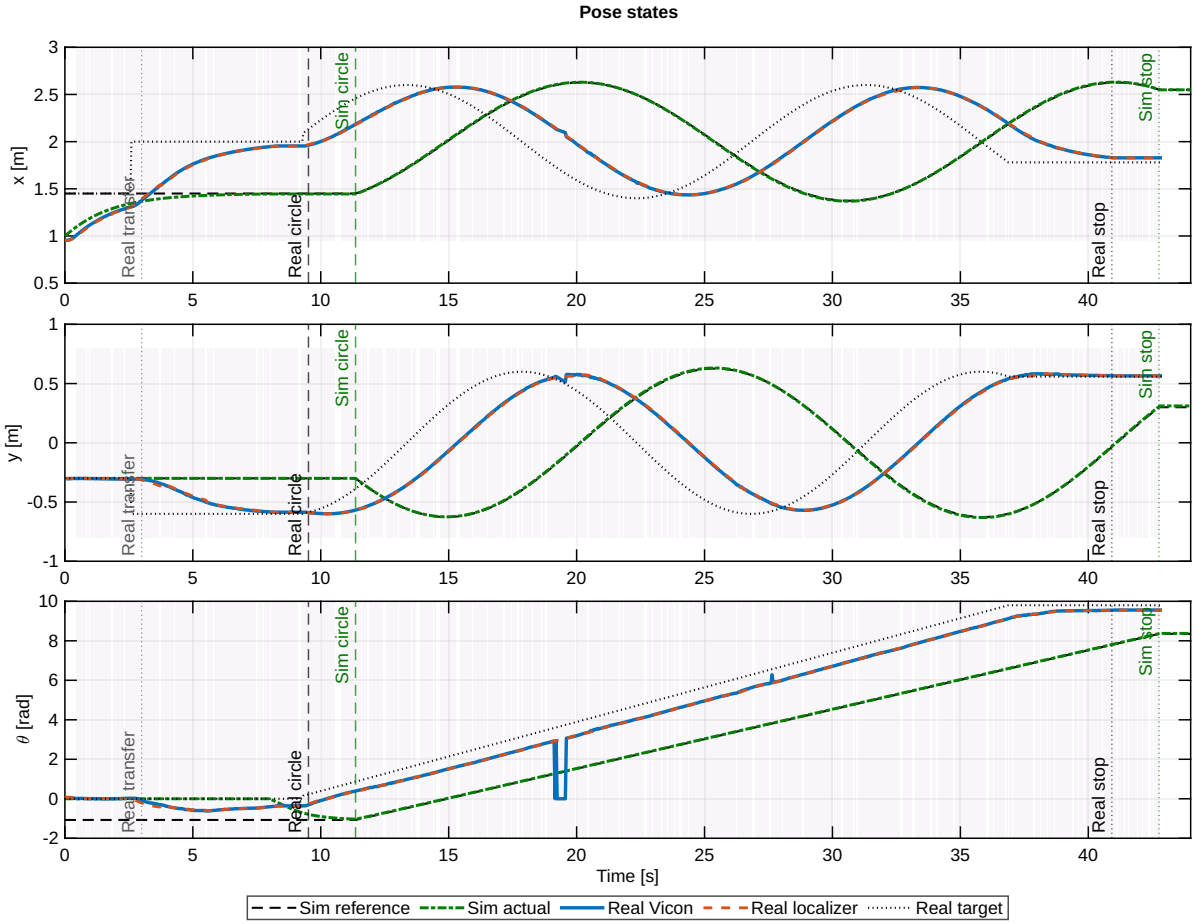


Figure 18: Task 3 pose states.

Figure 18 shows $x(t)$, $y(t)$, and $\theta(t)$. The real and simulated phase markers are not aligned because the two implementations use different transition logic and different circular references. Therefore, this figure is used only for qualitative comparison, not for sample-by-sample validation. The real Vicon and real localizer curves remain close, while the final simulation shows a smoother and more regular circular evolution. The visible discontinuity in the real θ curve is an angle wrapping/unwrapping effect in the plotted signal, not a physical jump of the robot.

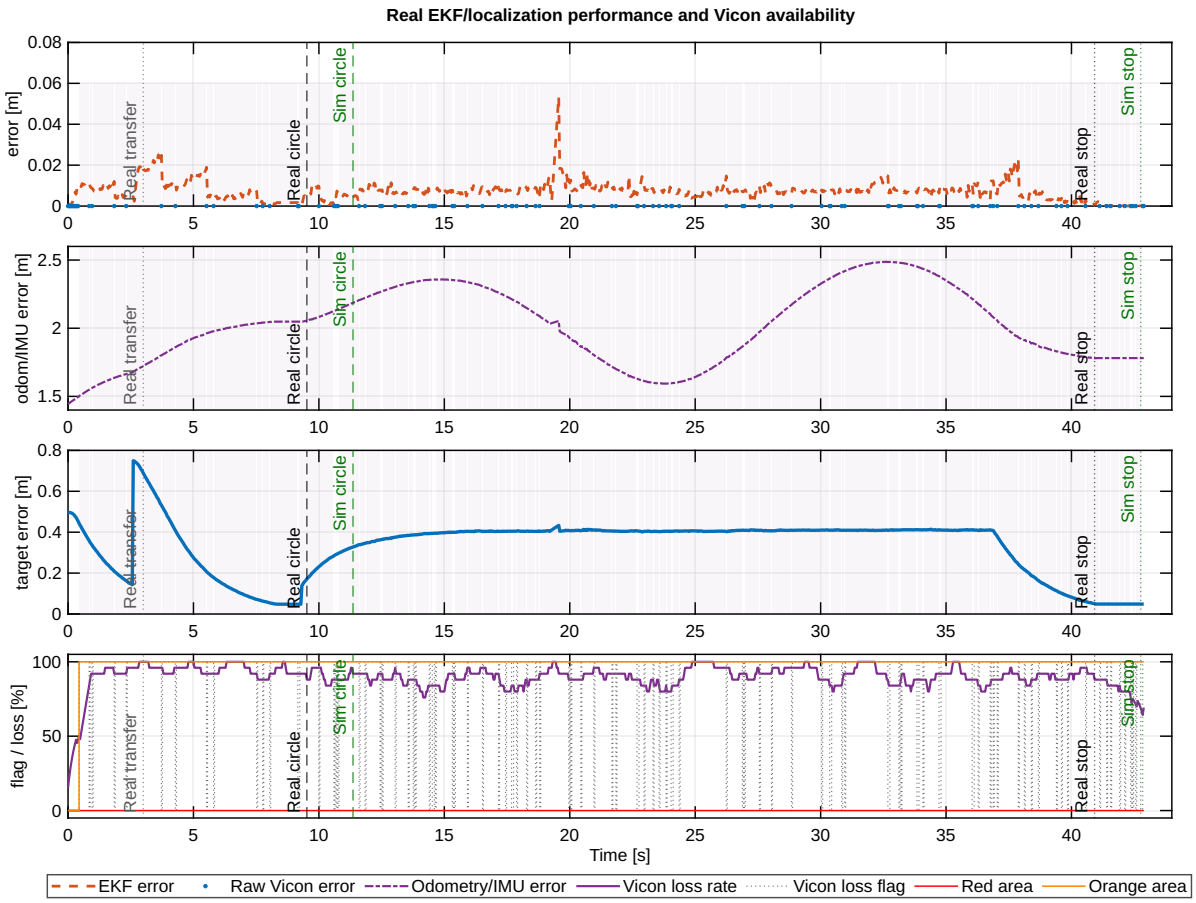


Figure 19: Task 3 localization and Vicon availability.

Figure 19 focuses on real localization and Vicon availability. The odometry/IMU estimate drifts strongly during the maneuver, with RMSE 2.04 m. In contrast, the EKF remains close to Vicon, with full-segment RMSE 0.0087 m and circle-only RMSE 0.0086 m. The Vicon loss percentage was 89.66%, and the red-area flag remained zero. This shows that the real run mainly tested EKF behavior under Vicon dropout in the orange region, and that pure odometry/IMU was not reliable for this maneuver.

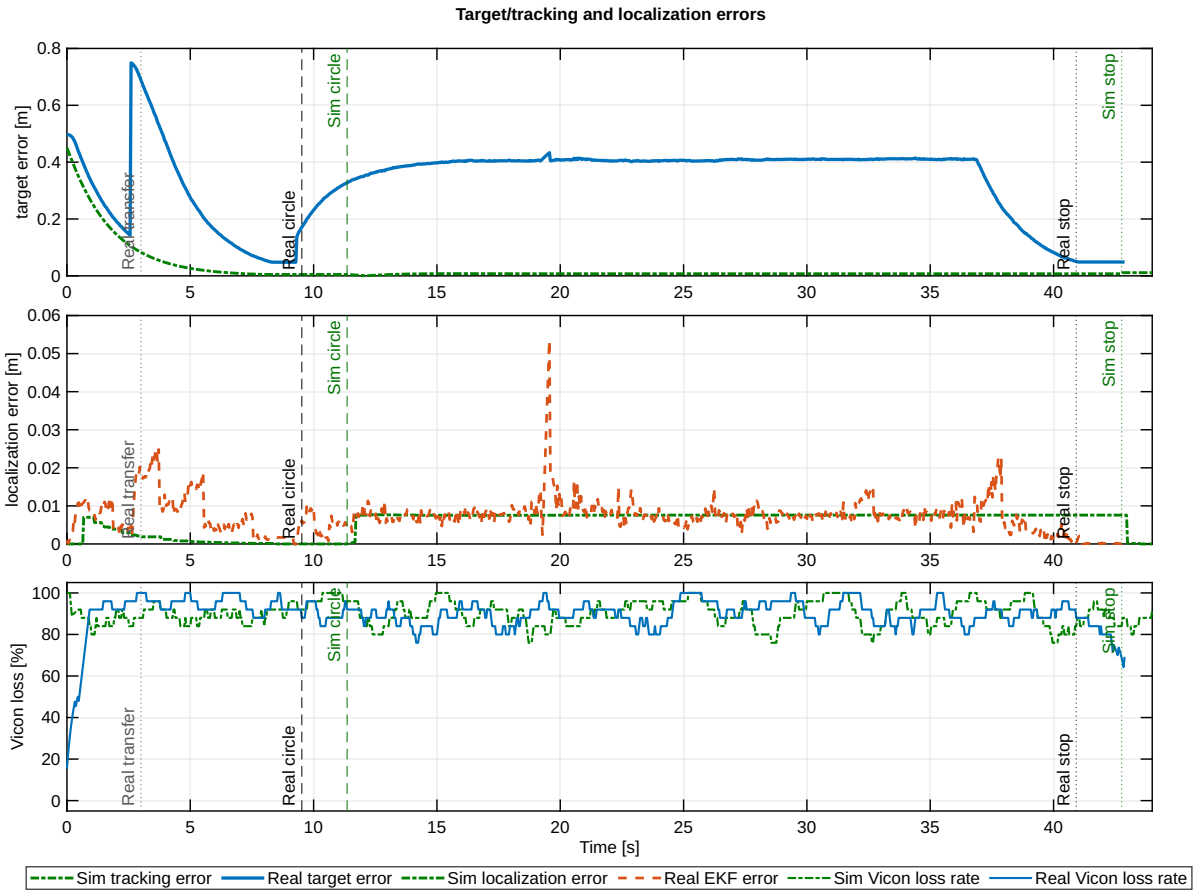


Figure 20: Task 3 tracking and localization errors.

Figure 20 must be interpreted carefully. The simulation tracking error is the valid quantitative tracking metric for the final tuned controller, because it compares the final simulated reference with the final simulated actual trajectory. The real target error is larger because the real data were generated with the earlier full-activity implementation and a different circular reference. Therefore, it should not be used to evaluate the final nonlinear tracking controller. The relevant real metric is the EKF localization error. One isolated EKF error peak of about 0.05 m appears near the middle of the circular motion. This is likely a short correction transient caused by intermittent Vicon availability or a temporarily inconsistent Vicon update in the orange region. Since the error quickly returns to the millimetric-centimetric range, it does not affect the overall localization conclusion.

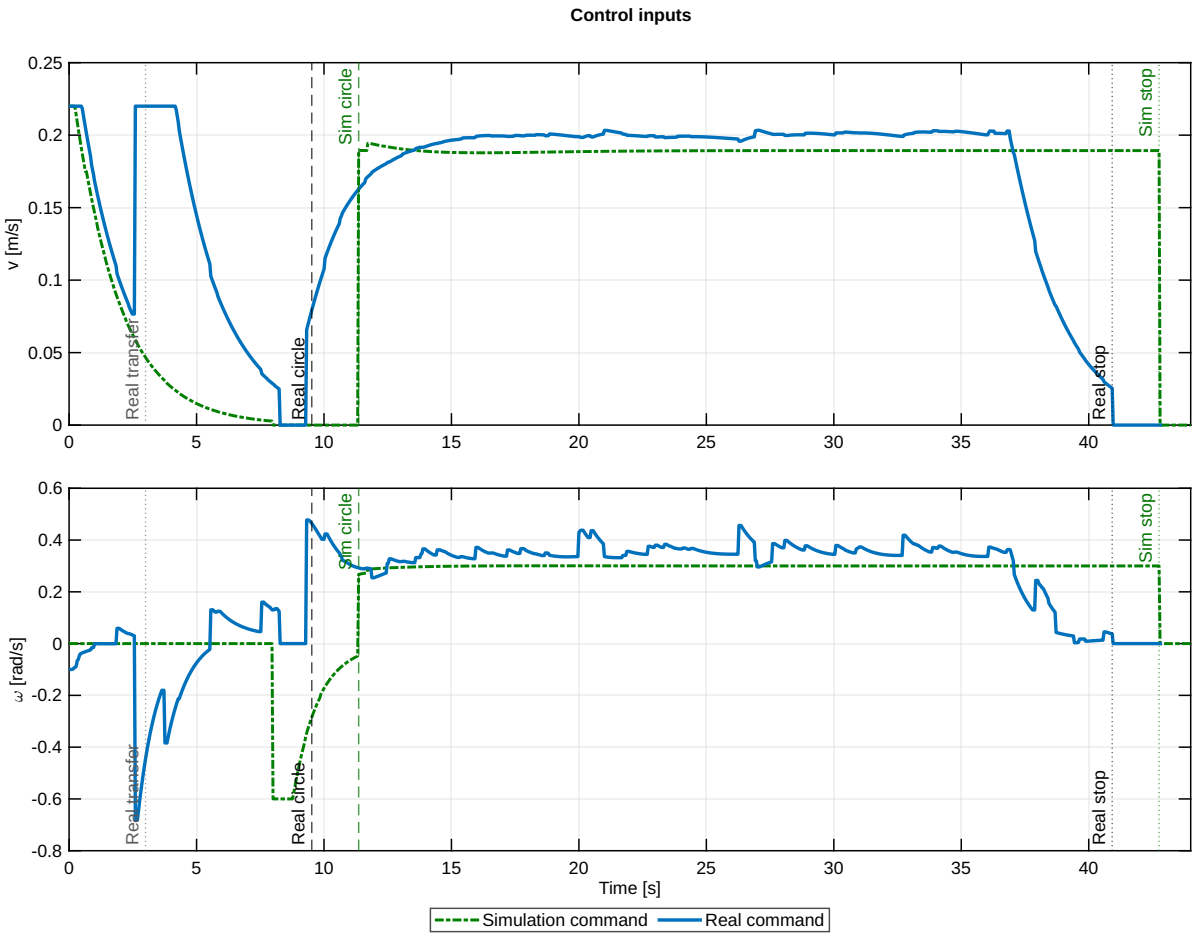


Figure 21: Task 3 control inputs.

Figure 21 compares the commanded unicycle inputs. In the final simulation, the circular-phase command becomes smooth and almost constant, as expected for a constant-speed circular reference. The nominal circular linear speed is

$$v_r = R\omega_c = 0.6265 \cdot 0.30 = 0.188 \text{ m/s},$$

which is below the imposed 0.22 m/s limit. The simulated angular velocity is approximately 0.30 rad/s during the circle. The real command is less smooth, especially during transfer and transition phases, because it comes from the earlier implementation and experimental feedback. Nevertheless, the maximum absolute real linear velocity remained 0.220 m/s, so the velocity constraint was respected.

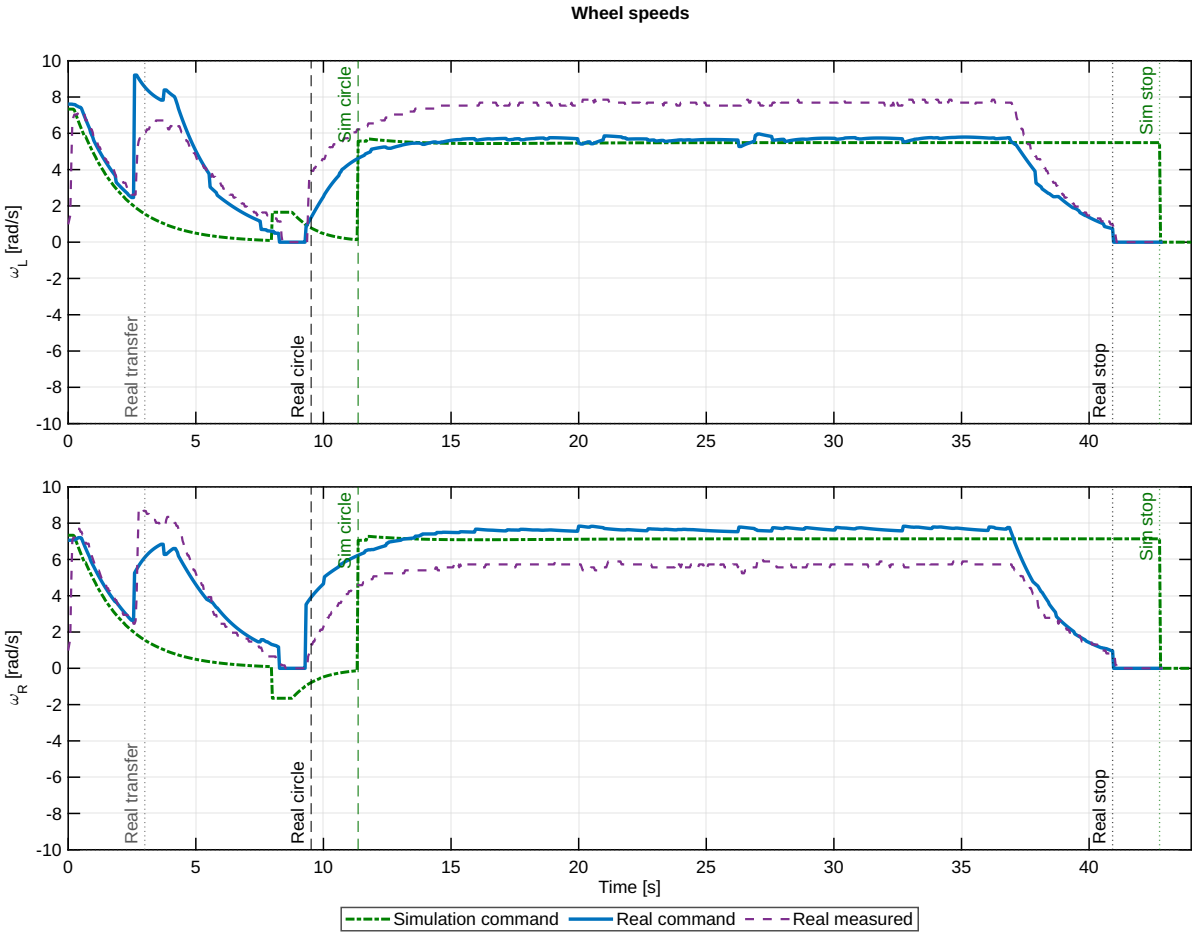


Figure 22: Task 3 wheel angular velocities.

Figure 22 verifies the wheel-speed constraint. In simulation, the maximum absolute commanded wheel speed was 7.33 rad/s. In the real run, the maximum absolute commanded wheel speed was 9.41 rad/s, and the maximum absolute measured wheel speed was 9.00 rad/s. Therefore, the command-level wheel-speed limit of 10 rad/s was respected in both cases. The difference between commanded and measured wheel speeds is expected on the real platform and can be attributed to low-level motor dynamics, wheel-ground interaction, calibration uncertainty, and measurement noise.

Overall, Task 3 was successful in simulation and experimentally meaningful on the real robot. The final simulation demonstrates the improved finite-state manager and nonlinear circular tracking controller. The real run shows that the robot can execute the maneuver around the cube, that the EKF remains accurate under frequent Vicon loss, and that the wheel-speed limits are respected. The main limitation is that the physical robot was not tested again after the final robust controller tuning. Therefore, the real run supports feasibility, collision-free motion around the cube, localization robustness, and actuator-limit verification, while the final simulation validates the tuned circular tracking performance.

Task 4

The objective of Task 4 was to repeat the circular motion around the cube without using the Vicon motion-capture system. This task was solved only in simulation; no real-robot validation was performed for Task 4. The robot starts close to the final configuration of Task 2 and follows the same circular reference used in Task 3. The feedback pose is not provided by Vicon, but by a LiDAR-

based Extended Kalman Filter (EKF) using wheel-encoder odometry, IMU yaw-rate information, and LiDAR range measurements.

The goal of this task was therefore not to validate a physical experiment, but to test whether the proposed LiDAR-EKF localization strategy can provide a bounded and usable pose estimate in simulation. Since no explicit sensor-noise values were provided for this task, the LiDAR, encoder, IMU, actuation, dropout, and initialization perturbations introduced here are artificial simulation assumptions. They were added only to avoid an idealized simulation and to test the EKF behavior under non-ideal sensing. They are not professor-provided sensor specifications.

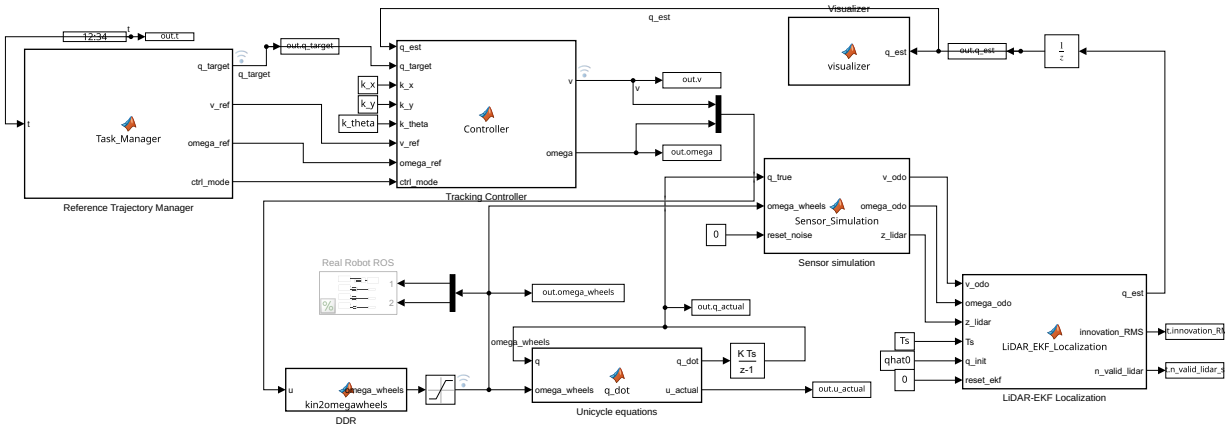


Figure 23: Task 4 Simulink implementation.

Figure 23 shows the Simulink model used for Task 4. The reference manager generates the approach phase and the circular reference around the cube. The tracking controller uses the EKF pose estimate as feedback and computes the linear and angular commands. These commands are converted into wheel-speed commands and applied to the simulated differential-drive robot. The sensor simulation block generates encoder, IMU, and LiDAR measurements, and the LiDAR-EKF block estimates the robot pose. The Vicon system is not used in the feedback loop. The Real Robot ROS block is kept only as an inactive placeholder inherited from the previous task models; it is not used in the Task 4 simulation results.

The robot pose is

$$q = \begin{bmatrix} x & y & \theta \end{bmatrix}^T.$$

The LiDAR is translated with respect to the TurtleBot kinematic center along the sagittal axis by

$$l = 0.076 \text{ m}.$$

Therefore, the LiDAR center in the inertial frame is

$$p_L = \begin{bmatrix} x \\ y \end{bmatrix} + l \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix}.$$

For the i-th beam, the global beam direction is

$$\phi_i = \theta + \theta_i.$$

In the simulation, 36 LiDAR beams were used, with

$$\theta_i = -\pi + (i-1)\frac{2\pi}{36}, \qquad i = 1, \ldots, 36.$$

The cube position and geometry are known. Therefore, the LiDAR is not used to detect an unknown object. Instead, each beam is intersected with the known cube walls, and the predicted distance is compared with the simulated LiDAR measurement:

$$d_i = h_i(q) + v_i,$$

where $h_i(q)$ is the ray-intersection distance and v_i is the simulated LiDAR disturbance. Beams that do not intersect the cube, are dropped, or do not pass the innovation check are not used in the EKF correction.

The EKF state is the planar robot pose,

$$\hat{q} = \begin{bmatrix} \hat{x} & \hat{y} & \hat{\theta} \end{bmatrix}^T.$$

The prediction step uses the unicycle model:

$$\hat{x}_{k|k-1} = \hat{x}_{k-1|k-1} + T_s v_k \cos\left(\hat{\theta}_{k-1|k-1} + \frac{T_s\omega_k}{2}\right),$$

$$\hat{y}_{k|k-1} = \hat{y}_{k-1|k-1} + T_s v_k \sin\left(\hat{\theta}_{k-1|k-1} + \frac{T_s\omega_k}{2}\right),$$

$$\hat{\theta}_{k|k-1} = \hat{\theta}_{k-1|k-1} + T_s\omega_k.$$

The angular odometry used in the prediction combines encoder and IMU yaw-rate information:

$$\omega_{\text{odo}} = 0.45\,\omega_{\text{enc}} + 0.55\,\omega_{\text{imu}}.$$

The LiDAR correction uses the nonlinear measurement model

$$y_k = h(q_k) + v_k.$$

The corresponding measurement matrix is

$$H_k = \left.\frac{\partial h}{\partial q}\right|_{\hat{q}_{k|k-1}}.$$

For each valid LiDAR beam, the measurement Jacobian was obtained analytically from the ray-wall intersection model. Let the LiDAR center be

$$p_L = \begin{bmatrix} x + l\cos\theta \\ y + l\sin\theta \end{bmatrix},$$

and let the global direction of beam i be

$$\phi_i = \theta + \theta_i.$$

The corresponding ray is parameterized as

$$p(s) = p_L + s \begin{bmatrix} \cos\phi_i \\ \sin\phi_i \end{bmatrix}, \qquad s \geq 0.$$

For a known wall described by

$$AX + BY + C = 0,$$

substitution of the ray equation into the wall equation gives

$$A(p_x + s\cos\phi_i) + B(p_y + s\sin\phi_i) + C = 0.$$

Defining

$$N(q) = A(x + l\cos\theta) + B(y + l\sin\theta) + C,$$

and

$$D(q) = A\cos(\theta + \theta_i) + B\sin(\theta + \theta_i),$$

the predicted LiDAR range for beam i is

$$h_i(q) = -\frac{N(q)}{D(q)}.$$

The required partial derivatives are obtained from

$$\frac{\partial N}{\partial x} = A, \qquad \frac{\partial N}{\partial y} = B, \qquad \frac{\partial N}{\partial \theta} = l(-A\sin\theta + B\cos\theta),$$

and

$$\frac{\partial D}{\partial x} = 0, \qquad \frac{\partial D}{\partial y} = 0, \qquad \frac{\partial D}{\partial \theta} = -A\sin(\theta + \theta_i) + B\cos(\theta + \theta_i).$$

Using the quotient rule, the analytical Jacobian row associated with beam i is

$$H_{k,i} = \left[-\frac{A}{D} \quad -\frac{B}{D} \quad \frac{N\left[-A\sin(\theta + \theta_i) + B\cos(\theta + \theta_i)\right] - l(-A\sin\theta + B\cos\theta)D}{D^2} \right],$$

evaluated at the predicted pose $\hat{q}_{k|k-1}$. This row is used only for beams that produce a valid forward intersection with one of the known cube walls and pass the implemented validity and innovation checks.

The simulation sampling time was $T_s = 0.04$ s. The artificial sensor perturbations were: LiDAR range-noise standard deviation 0.006 m, LiDAR dropout probability 3%, encoder increment-noise standard deviation 0.0008 rad, and IMU yaw-rate noise standard deviation 0.010 rad/s. The EKF process covariance was

$$Q = \mathrm{diag}(1.5\times10^{-5},\ 1.5\times10^{-5},\ 6\times10^{-5}).$$

For the exact-initialization case, the initial covariance was

$$P_0 = \mathrm{diag}(10^{-4},\ 10^{-4},\ (1^\circ)^2),$$

while for the wrong-EKF-initialization case it was

$$P_0 = \mathrm{diag}(5\times10^{-3},\ 5\times10^{-3},\ (7^\circ)^2).$$

The LiDAR update used innovation gating. In the exact case, a stricter gate of 0.045 m, a minimum of 4 valid rays, and LiDAR variance $(0.010)^2$ were used. In the wrong-EKF-initialization case, the gate was 0.120 m at the beginning and 0.080 m afterwards, with a minimum of 3 valid rays and LiDAR variance $(0.006)^2$. These values are EKF simulation-tuning parameters, not real sensor specifications. The EKF measurement variance is therefore a tuning value used by the filter and

does not necessarily coincide exactly with the standard deviation used to perturb the simulated LiDAR measurements.

Two simulations were compared. In the exact-initialization case, the simulated robot and the EKF start from the same initial pose. In the wrong-EKF-initialization case, the simulated robot starts from

$$q_{\text{true},0} = \begin{bmatrix} 0.960 & -0.340 & 4.00^\circ \end{bmatrix}^T,$$

while the EKF is intentionally initialized from

$$\hat{q}_0 = \begin{bmatrix} 1.010 & -0.390 & 9.00^\circ \end{bmatrix}^T.$$

Thus, the initial tracking error is zero, because the robot starts on the nominal reference, but the initial localization position error is 0.0707 m and the initial localization heading error is 5.00°. This simulation directly evaluates localization-initialization uncertainty, i.e. whether the LiDAR-EKF can recover from a wrong initial pose estimate when Vicon is unavailable.

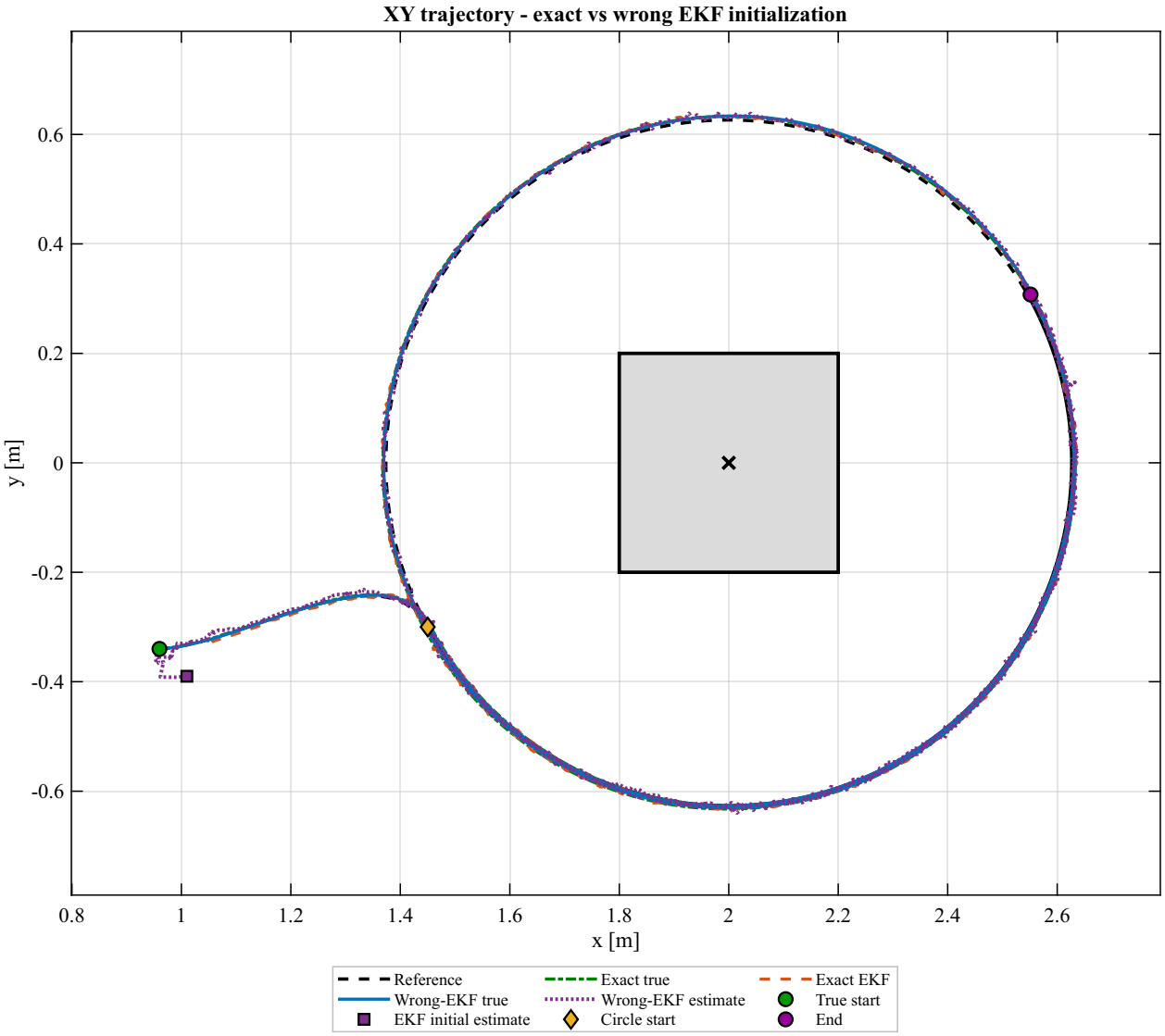


Figure 24: Task 4 x-y trajectory.

Figure 24 compares the reference, simulated true trajectories, and EKF estimates. In the wrong-EKF-initialization case, the true robot start and the EKF initial estimate are marked separately, showing the imposed initial localization error. Despite this wrong EKF initialization, the estimated

trajectory rapidly approaches the simulated true trajectory and remains close to it during the circular motion around the cube. This confirms that the LiDAR correction provides useful pose information in the simulated Vicon-free condition.

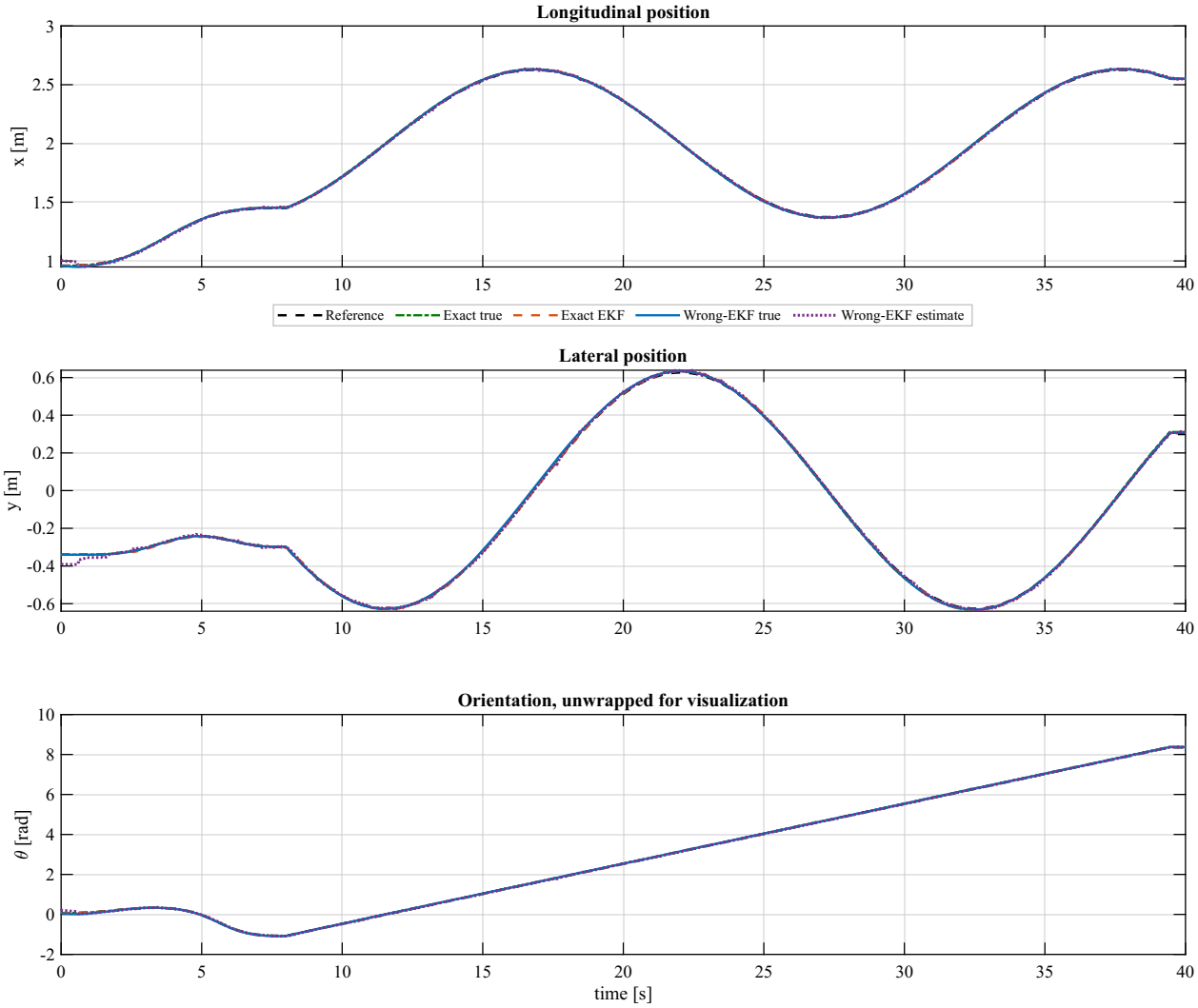


Figure 25: Task 4 pose states.

Figure 25 shows $x(t)$, $y(t)$, and $\theta(t)$. In the exact case, the EKF estimate almost overlaps the simulated true pose from the beginning. In the wrong-EKF case, the initial difference between the estimated pose and the true pose is visible at the start, but the estimate quickly aligns with the simulated true state. The orientation is shown unwrapped only for visualization; the EKF and controller internally use wrapped angles.

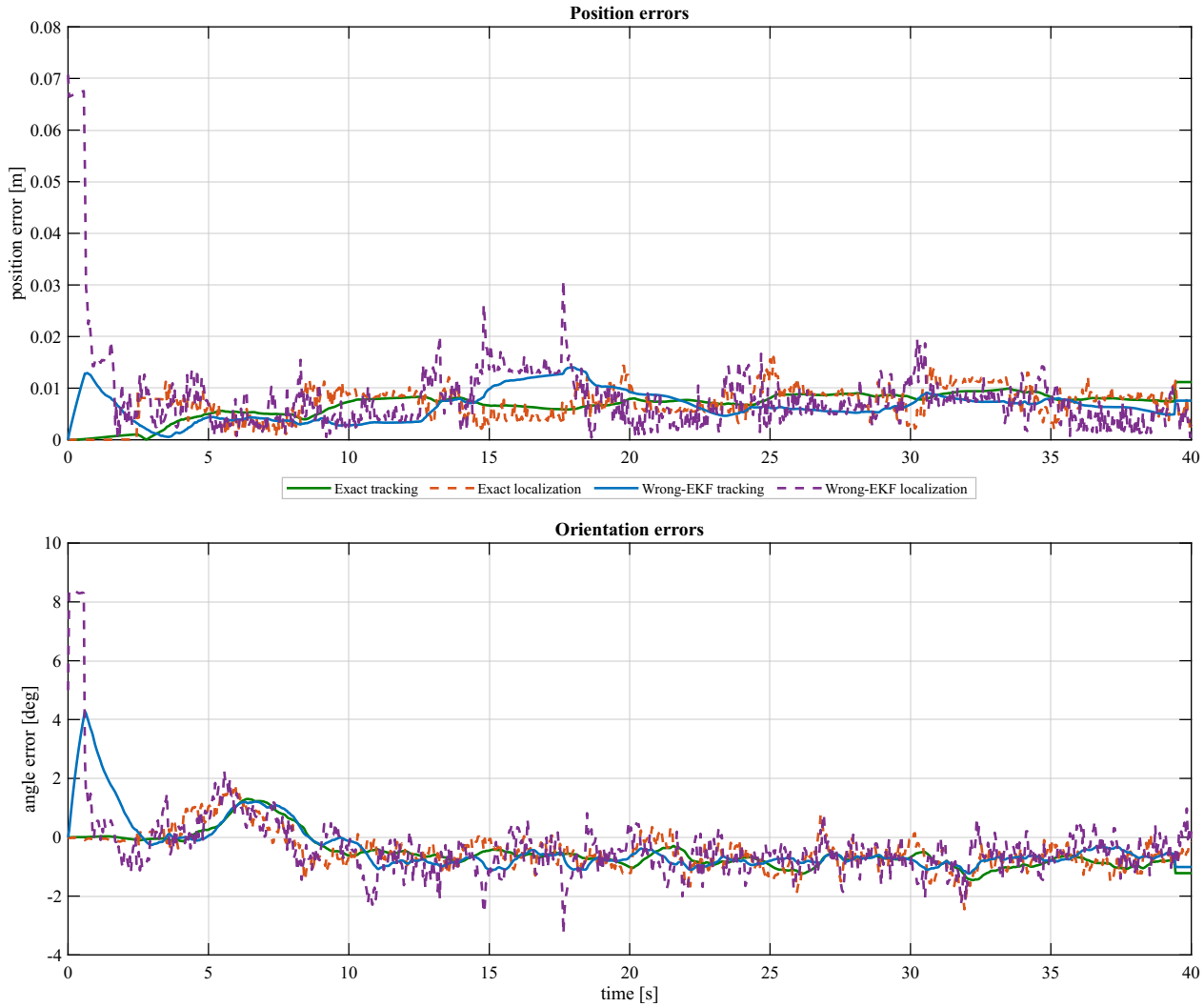


Figure 26: Task 4 tracking and localization errors.

Figure 26 shows the tracking and localization errors. In the exact case, the initial localization error is zero. In the wrong-EKF case, the initial localization position error is 0.0707 m, corresponding to the imposed EKF initialization offset. The error decreases quickly after the first LiDAR corrections. Using the criterion that the localization position error must remain below 0.02 m for at least 1 s, the wrong-EKF case converges after 0.84 s. The localization RMSE is 0.0076 m in the exact case and 0.0120 m in the wrong-EKF case. The final localization errors are 0.0018 m and 0.0029 m, respectively. Therefore, the wrong initialization increases the transient error but does not prevent convergence in simulation.

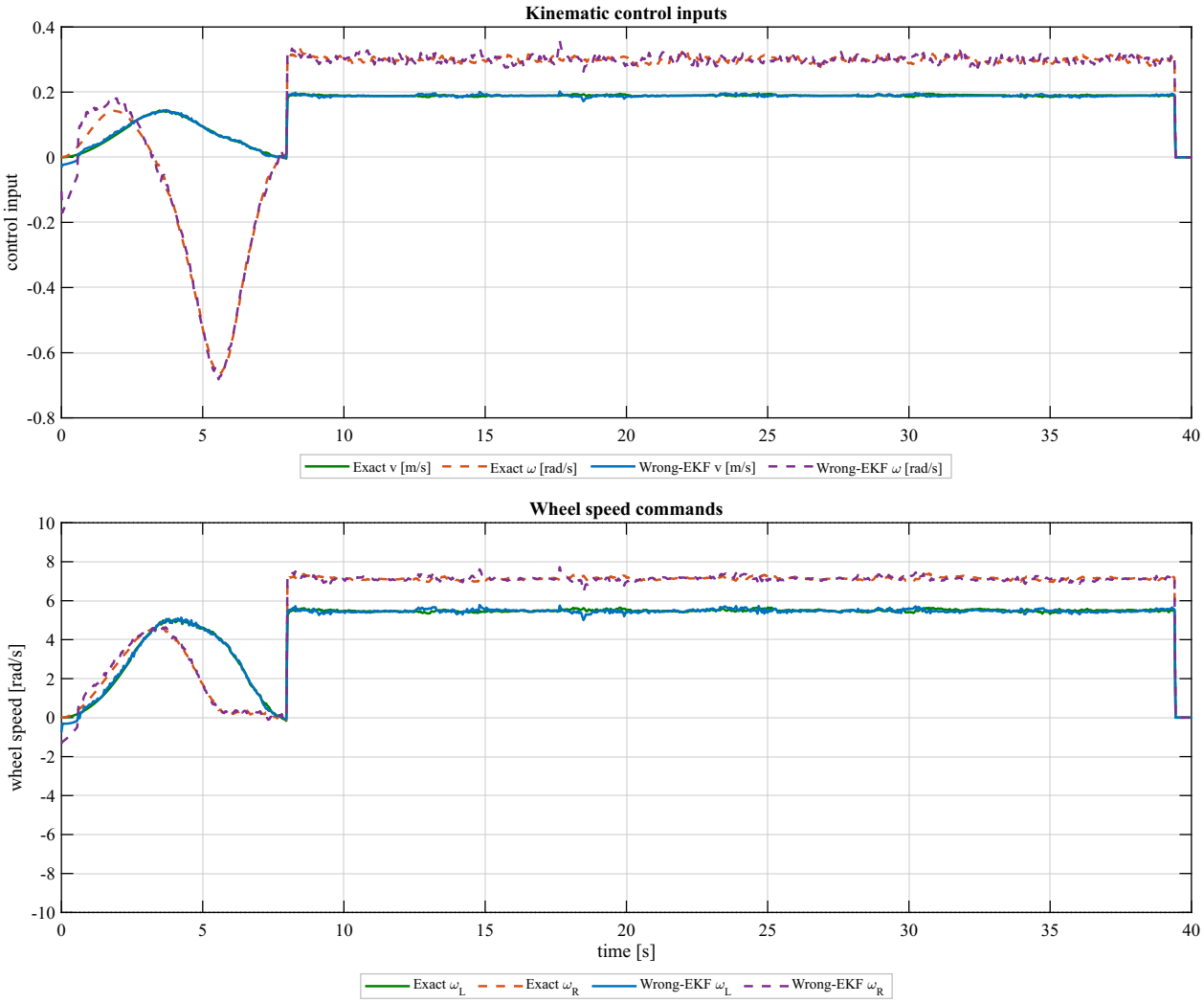


Figure 27: Task 4 control inputs and wheel speeds.

Figure 27 reports the commanded linear and angular velocities and the corresponding wheel speeds. The wheel-speed limit was 10 rad/s. The maximum commanded wheel speed was 7.4042 rad/s in the exact case and 7.7267 rad/s in the wrong-EKF case. Therefore, even with the wrong EKF initialization, the simulated control commands remain within the admissible actuator range. The slightly larger wheel-speed demand in the wrong-EKF case is consistent with the initial correction transient caused by the wrong pose estimate.

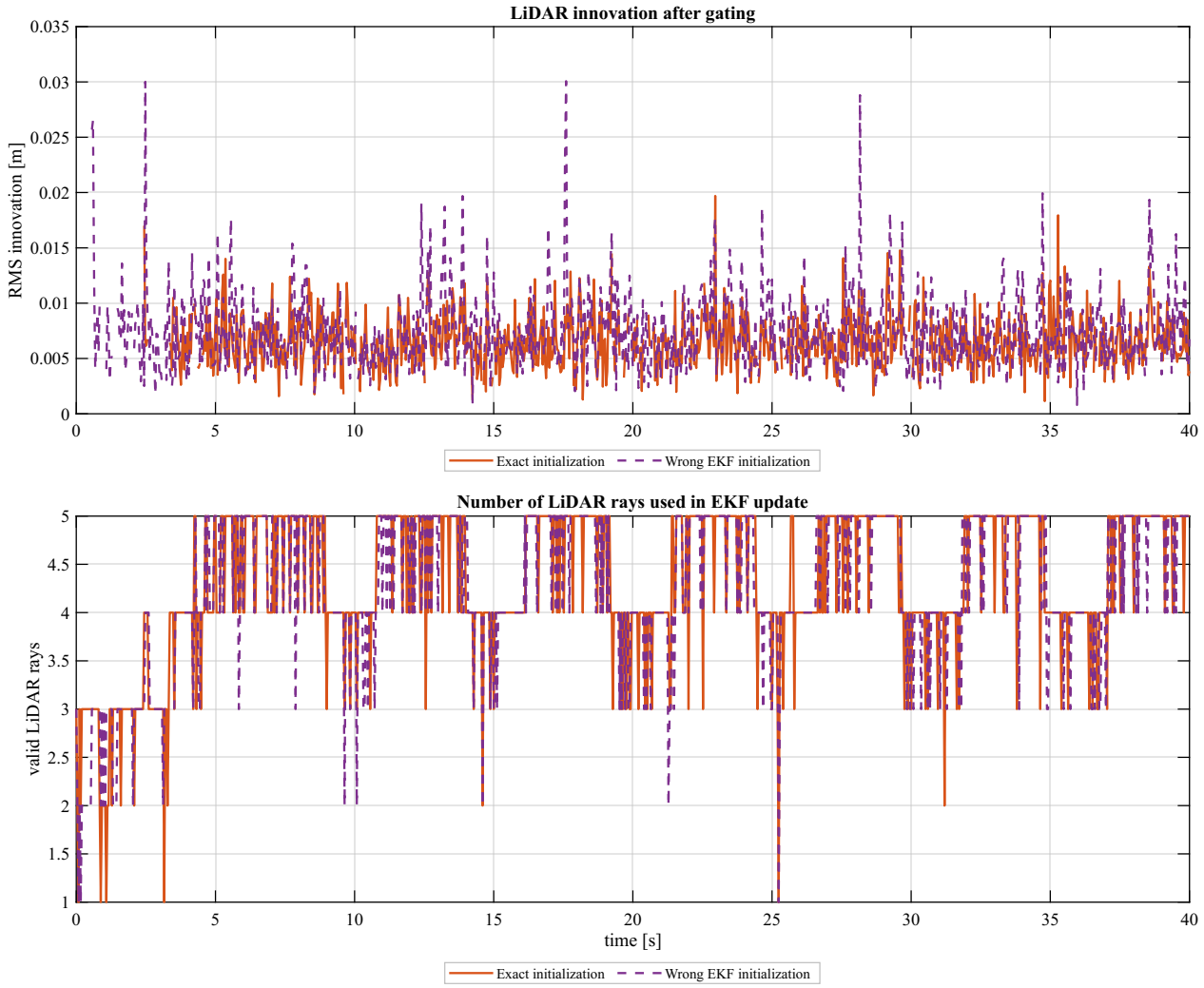


Figure 28: Task 4 LiDAR innovation and valid rays.

Figure 28 shows the LiDAR innovation and the number of valid rays used by the EKF. The RMS innovation remains bounded in both simulations. The maximum innovation is 0.0197 m in the exact case and 0.0301 m in the wrong-EKF case. The larger peaks in the wrong-EKF case are expected because the initial pose estimate produces less accurate predicted LiDAR ranges before convergence. The mean numbers of valid rays are similar in the two cases, 4.31 and 4.30, respectively. The valid-update percentages are 86.81% in the exact case and 97.30% in the wrong-EKF case; the higher value in the wrong-EKF case is consistent with the looser initial innovation gate and the lower minimum number of valid rays used for that simulation.

Over the reported 40.00 s interval, the tracking RMSE was 0.0072 m in the exact case and 0.0071 m in the wrong-EKF case. The localization RMSE increased from 0.0076 m to 0.0120 m due to the wrong EKF initialization, but the final localization error remained small. Overall, Task 4 was solved in simulation only. The results show that, in simulation, the LiDAR-based EKF with the analytical measurement Jacobian can recover from an initial pose-estimation error and provide the feedback pose when Vicon is unavailable during the circular motion around the known cube. However, real-robot validation would still be required before claiming an experimental replacement of the motion-capture system.